

TAREA_CHRISTOPHER

April 28, 2026

Tarea Christopher Lagos

ICM

```
[1]: # PRELIMINARES
# CARGA DE BASE DE DATOS
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

# 1. Definir la ruta de forma robusta
# Si el CSV está en la misma carpeta que tu script (.py o .ipynb):
file_name = 'student_productivity.csv'

# Verificamos si el archivo existe en el directorio actual de trabajo
if os.path.exists(file_name):
    try:
        # Carga de la base de datos
        df = pd.read_csv(file_name)
        print(f" Archivo cargado exitosamente: {file_name}")
        print(f" Dimensiones: {df.shape[0]} filas y {df.shape[1]} columnas.")

        # Vista rápida de los datos
        print("\nPrimeras filas:")
        print(df.head())

    except Exception as e:
        print(f" Ocurrió un error al leer el archivo: {e}")
else:
    print(f" Error: El archivo '{file_name}' no se encuentra en la carpeta_
↪ actual.")
    print(f" Directorio actual: {os.getcwd()}")

# 2. Configuración estética para gráficos
sns.set_theme(style="whitegrid") # Actualizado de set_context para mejores_
↪ visuales
```

```
plt.rcParams['figure.figsize'] = (10, 6)
```

Archivo cargado exitosamente: student_productivity.csv

Dimensiones: 5621 filas y 22 columnas.

Primeras filas:

	student_id	age	gender	academic_level	study_hours	self_study_hours	\
0	1	20.0	NaN	Undergraduate	5.37	2.09	
1	2	16.0	Female	High School	5.85	5.04	
2	3	18.0	Female	Undergraduate	5.69	2.27	
3	4	24.0	Male	Undergraduate	2.32	1.06	
4	5	24.0	Female	Postgraduate	3.87	2.63	

	online_classes_hours	social_media_hours	gaming_hours	sleep_hours	...	\
0	1.85	3.66	2.32	7.73	...	
1	1.87	3.60	2.79	6.11	...	
2	0.00	2.93	3.94	7.11	...	
3	2.30	4.34	2.37	8.54	...	
4	2.54	3.71	1.30	7.69	...	

	caffeine_intake_mg	part_time_job	upcoming_deadline	internet_quality	\
0	475.0	No	0.0	Good	
1	362.0	no	0.0	Good	
2	200.0	No	0.0	Average	
3	233.0	No	1.0	Poor	
4	159.0	No	1.0	Poor	

	mental_health_score	drug_use	focus_index	burnout_level	\
0	3.0	NaN	19.01	31.77	
1	10.0	NaN	42.10	45.89	
2	5.0	NaN	21.93	37.07	
3	3.0	1.0	13.47	43.63	
4	2.0	NaN	19.95	56.62	

	productivity_score	exam_score
0	42.59	25.08
1	67.15	37.83
2	37.68	18.66
3	12.83	1.00
4	18.53	7.78

[5 rows x 22 columns]

```
[2]: # visualizacion de los datos
# Verificación de tipos de datos
print("--- Tipos de Datos por Columna ---")
print(df.dtypes)
```

```
# Visualización de las primeras filas para inspección visual
df.head()
```

```
--- Tipos de Datos por Columna ---
```

```
student_id      int64
age             float64
gender          object
academic_level  object
study_hours     float64
self_study_hours float64
online_classes_hours float64
social_media_hours float64
gaming_hours    float64
sleep_hours     float64
screen_time_hours float64
exercise_minutes object
caffeine_intake_mg float64
part_time_job   object
upcoming_deadline float64
internet_quality object
mental_health_score float64
drug_use        float64
focus_index    float64
burnout_level   float64
productivity_score float64
exam_score      float64
dtype: object
```

```
[2]:  student_id  age  gender  academic_level  study_hours  self_study_hours  \
0          1  20.0   NaN    Undergraduate      5.37          2.09
1          2  16.0 Female    High School      5.85          5.04
2          3  18.0 Female    Undergraduate      5.69          2.27
3          4  24.0   Male    Undergraduate      2.32          1.06
4          5  24.0 Female    Postgraduate      3.87          2.63
```

```
online_classes_hours  social_media_hours  gaming_hours  sleep_hours  ...  \
0                1.85                3.66          2.32          7.73  ...
1                1.87                3.60          2.79          6.11  ...
2                0.00                2.93          3.94          7.11  ...
3                2.30                4.34          2.37          8.54  ...
4                2.54                3.71          1.30          7.69  ...
```

```
caffeine_intake_mg  part_time_job  upcoming_deadline  internet_quality  \
0             475.0             No             0.0             Good
1             362.0             no             0.0             Good
2             200.0             No             0.0             Average
```

3	233.0	No	1.0	Poor
4	159.0	No	1.0	Poor

	mental_health_score	drug_use	focus_index	burnout_level \
0	3.0	NaN	19.01	31.77
1	10.0	NaN	42.10	45.89
2	5.0	NaN	21.93	37.07
3	3.0	1.0	13.47	43.63
4	2.0	NaN	19.95	56.62

	productivity_score	exam_score
0	42.59	25.08
1	67.15	37.83
2	37.68	18.66
3	12.83	1.00
4	18.53	7.78

[5 rows x 22 columns]

```
[3]: #PREGUNTA 1
#LIMPIEZA DE LA BASE DE DATOS
import pandas as pd
import numpy as np

# 1. EXCLUIR CASOS (Umbral > 15 NaNs)
# Evaluamos nulos sin considerar 'exam_score' para no sesgar la variable 'Exam'
columnas_estudio = [col for col in df.columns if col not in ['exam_score', '
↳ 'student_id']]
df_limpio = df[df[columnas_estudio].isnull().sum(axis=1) <= 15].copy()

# 2. VARIABLE INDICADORA E INGENIERÍA
# Variable 'Exam': 0 si la nota es 1.0 (no rindió), 1 en caso contrario
df_limpio['Exam'] = np.where(df_limpio['exam_score'] > 1.0, 1, 0)

# Dummy "gaming_hours_Index"
df_limpio['gaming_hours_Index'] = np.where(df_limpio['gaming_hours'] > 0, 1, 0)

# Dummy "online_classes_hours_index"
df_limpio['online_classes_hours_index'] = np.
↳ where(df_limpio['online_classes_hours'] > 0, 1, 0)

# Transformación de ejercicio (Minutos a Horas y Dummy Index)
df_limpio['exercise_minutes'] = pd.to_numeric(df_limpio['exercise_minutes'],
↳ errors='coerce')
df_limpio['exercise_hours'] = df_limpio['exercise_minutes'] / 60
df_limpio['exercise_hours_index'] = np.where(df_limpio['exercise_hours'] > 0,
↳ 1, 0)
```

```

# Eliminamos la original de minutos
df_limpio = df_limpio.drop(columns=['exercise_minutes'])

# 4. IMPUTACIÓN Y NORMALIZACIÓN
# 4.3) Quitar drug_use por exceso de NAs
if 'drug_use' in df_limpio.columns:
    df_limpio = df_limpio.drop(columns=['drug_use'])

# 4.1, 4.2, 4.5) Normalización de categóricas y corregir duplicados
cats = ['gender', 'academic_level', 'part_time_job']
for col in cats:
    if col in df_limpio.columns:
        # Quitar espacios, estandarizar mayúsculas (soluciona Yes/yes,
        ↪ Undergraduate/undergraduate)
        df_limpio[col] = df_limpio[col].astype(str).str.strip().str.capitalize()
        # Rellenar vacíos con 'Other'
        df_limpio[col] = df_limpio[col].replace(['Nan', 'None', 'Null', ''],
        ↪ 'Other')

# 4.4) Imputación de numéricas con la media
num_cols = df_limpio.select_dtypes(include=[np.number]).columns
for col in num_cols:
    if col not in ['student_id', 'exam_score', 'Exam']:
        df_limpio[col] = df_limpio[col].fillna(df_limpio[col].mean())

# RESULTADOS FINALES
filas_finales, columnas_finales = df_limpio.shape

print("="*60)
print("PROCESAMIENTO COMPLETO DE BASE DE DATOS")
print("="*60)
print(f"Cantidad de FILAS finales:    {filas_finales}")
print(f"Cantidad de COLUMNAS finales: {columnas_finales}")
print("-" * 60)
print("Distribución de la variable 'Exam' (Asistencia):")
print(df_limpio['Exam'].value_counts())
print("="*60)

# Guardamos como la nueva base limpia
df_final = df_limpio

```

```

=====
PROCESAMIENTO COMPLETO DE BASE DE DATOS
=====

```

```

Cantidad de FILAS finales:    5193

```

Cantidad de COLUMNAS finales: 25

Distribución de la variable 'Exam' (Asistencia):

1 4416

0 777

Name: Exam, dtype: int64
=====

```
[4]: #MATRIZ DE CORRELACION
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# 1. Seleccionamos solo las variables numéricas para la correlación
# Incluimos las dummies y los índices que creamos
df_corr = df_limpio.select_dtypes(include=[np.number])

# 2. Calcular la matriz de correlación
matriz_corr = df_corr.corr()

# 3. Configuración del gráfico (Heatmap)
plt.figure(figsize=(16, 10))

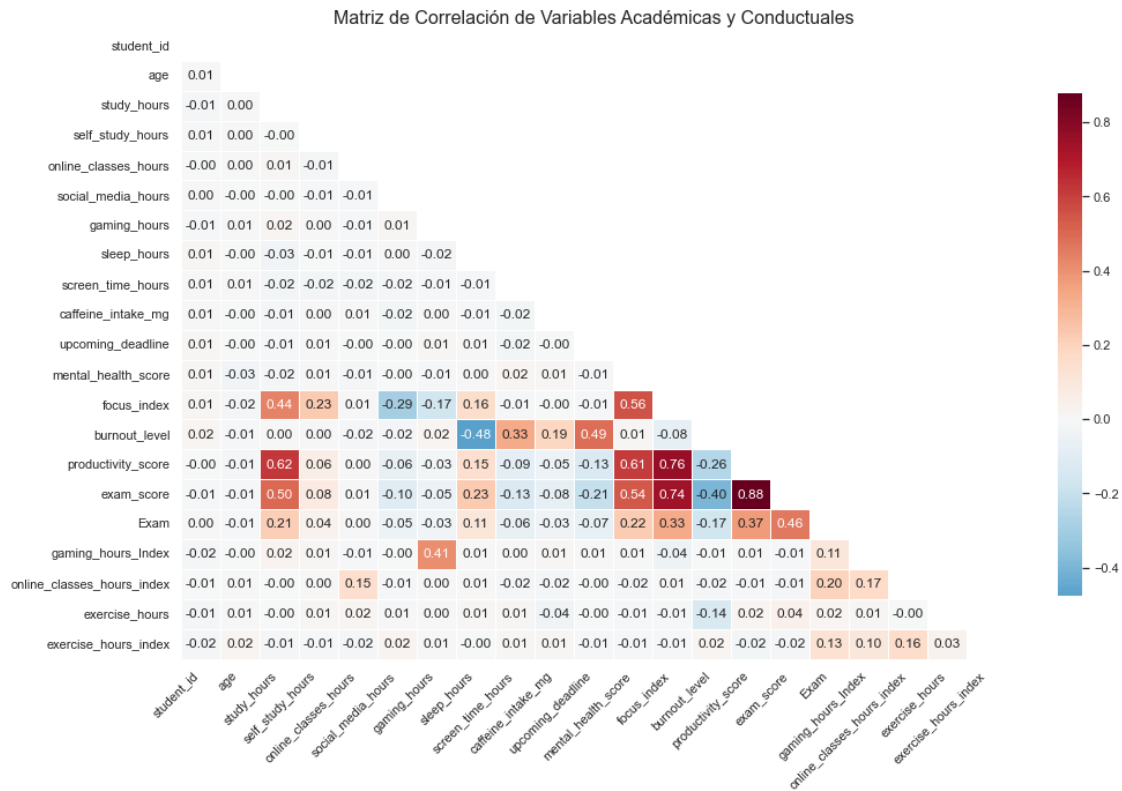
# Creamos una máscara para ver solo la mitad inferior (opcional, evita
↳ duplicados)
mask = np.triu(np.ones_like(matriz_corr, dtype=bool))

# Dibujar el heatmap
sns.heatmap(matriz_corr,
            mask=mask,
            annot=True,          # Muestra los números dentro de los cuadros
            fmt=".2f",           # Dos decimales
            cmap='RdBu_r',       # Color: Rojo (negativo) a Azul (positivo)
            center=0,            # El blanco es el 0
            linewidths=0.5,
            cbar_kws={"shrink": .8})

plt.title('Matriz de Correlación de Variables Académicas y Conductuales',
↳ fontsize=16)
plt.xticks(rotation=45, ha='right')
plt.show()

# 4. Mostrar las correlaciones más altas con la variable "Exam"
print("="*60)
print("CORRELACIÓN DE LAS VARIABLES CON LA ASISTENCIA (EXAM)")
print("="*60)
```

```
print(matriz_corr['Exam'].sort_values(ascending=False))
```



CORRELACIÓN DE LAS VARIABLES CON LA ASISTENCIA (EXAM)

```
Exam 1.000000
exam_score 0.460819
productivity_score 0.370930
focus_index 0.327737
mental_health_score 0.221564
study_hours 0.213909
online_classes_hours_index 0.203434
exercise_hours_index 0.131899
gaming_hours_Index 0.106942
sleep_hours 0.105735
self_study_hours 0.036543
exercise_hours 0.016543
online_classes_hours 0.004653
student_id 0.003446
age -0.008652
gaming_hours -0.030949
caffeine_intake_mg -0.032612
```

```

social_media_hours      -0.046819
screen_time_hours       -0.059265
upcoming_deadline       -0.072060
burnout_level           -0.167225
Name: Exam, dtype: float64

```

```

[5]: #PREGUNTA 2.1
#Modelo MCO COMPLETO
import statsmodels.api as sm
import pandas as pd

# 1. Preparación de variables categóricas
# Convertimos variables como 'gender' y 'academic_level' en dummies numéricas
df_mco = pd.get_dummies(df_limpio, drop_first=True)

# 2. Definición de variables
# Excluimos 'exam_score' (porque define a Exam) y 'student_id'
X = df_mco.drop(columns=['Exam', 'exam_score', 'student_id'], errors='ignore')
y = df_mco['Exam']

# 3. Asegurar que todos los datos sean flotantes para evitar errores de cálculo
X = X.astype(float)

# 4. Añadir la constante (Intercepto)
X = sm.add_constant(X)

# 5. Ajuste del modelo MCO (OLS)
modelo_asistencia = sm.OLS(y, X).fit()

# 6. Mostrar el resumen de resultados
print(modelo_asistencia.summary())

```

OLS Regression Results

```

=====
Dep. Variable:          Exam    R-squared:                0.266
Model:                  OLS    Adj. R-squared:            0.262
Method:                 Least Squares    F-statistic:        69.39
Date:                  mié., 22 abr. 2026    Prob (F-statistic):    1.39e-321
Time:                  10:11:37    Log-Likelihood:       -1211.8
No. Observations:      5193    AIC:                  2480.
Df Residuals:          5165    BIC:                  2663.
Df Model:               27
Covariance Type:       nonrobust
=====
=====

```

	coef	std err	t	P> t
[0.025	0.975]			

```

-----

```

const		0.4698	0.067	7.017	0.000
0.339	0.601				
age		-0.0013	0.002	-0.872	0.383
-0.004	0.002				
study_hours		0.0008	0.005	0.185	0.853
-0.008	0.010				
self_study_hours		-0.0009	0.004	-0.224	0.823
-0.009	0.007				
online_classes_hours		-0.0038	0.005	-0.832	0.405
-0.013	0.005				
social_media_hours		0.0007	0.003	0.206	0.837
-0.006	0.007				
gaming_hours		-0.0050	0.005	-1.101	0.271
-0.014	0.004				
sleep_hours		0.0036	0.005	0.666	0.505
-0.007	0.014				
screen_time_hours		-0.0018	0.002	-0.847	0.397
-0.006	0.002				
caffeine_intake_mg		-2.12e-05	3.27e-05	-0.648	0.517
-8.53e-05	4.29e-05				
upcoming_deadline		0.0019	0.013	0.149	0.882
-0.023	0.027				
mental_health_score		0.0002	0.003	0.060	0.952
-0.006	0.006				
focus_index		0.0045	0.001	5.225	0.000
0.003	0.006				
burnout_level		-0.0020	0.001	-3.046	0.002
-0.003	-0.001				
productivity_score		0.0055	0.001	7.615	0.000
0.004	0.007				
gaming_hours_Index		0.0274	0.013	2.040	0.041
0.001	0.054				
online_classes_hours_index		0.1180	0.018	6.644	0.000
0.083	0.153				
exercise_hours		-0.0012	0.007	-0.172	0.864
-0.014	0.012				
exercise_hours_index		0.0476	0.012	3.953	0.000
0.024	0.071				
gender_Male		0.0070	0.009	0.750	0.453
-0.011	0.025				
gender_Other		-0.0341	0.013	-2.682	0.007
-0.059	-0.009				
academic_level_Other		-0.2620	0.024	-10.839	0.000
-0.309	-0.215				
academic_level_Postgraduate		0.0107	0.011	1.001	0.317
-0.010	0.032				
academic_level_Undergraduate		0.0329	0.011	3.060	0.002

0.012	0.054				
part_time_job_Other		-0.1939	0.020	-9.489	0.000
-0.234	-0.154				
part_time_job_Yes		-0.0056	0.011	-0.514	0.607
-0.027	0.016				
internet_quality_Good		0.0233	0.010	2.243	0.025
0.003	0.044				
internet_quality_Poor		0.0117	0.010	1.119	0.263
-0.009	0.032				

Omnibus:	1096.211	Durbin-Watson:	2.028
Prob(Omnibus):	0.000	Jarque-Bera (JB):	2014.038
Skew:	-1.324	Prob(JB):	0.00
Kurtosis:	4.515	Cond. No.	4.67e+03

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 4.67e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```
[6]: #PREGUNTA 2.2
#MODELO MCO SIGNIFICATIVO
import statsmodels.api as sm
import pandas as pd

# 1. Lista de variables originales que resultaron significativas (o contienen
#    categorías significativas)
vars_originales_sig = [
    'focus_index',
    'burnout_level',
    'productivity_score',
    'gaming_hours_Index',
    'online_classes_hours_index',
    'exercise_hours_index',
    'gender',          # Contiene gender_Other
    'academic_level',  # Contiene Undergraduate y Other
    'part_time_job',   # Contiene Other
    'internet_quality' # Contiene Good
]

# 2. Creamos un nuevo DataFrame solo con estas variables + la dependiente
df_reducido = df_limpio[vars_originales_sig + ['Exam']].copy()

# 3. Convertimos las categóricas en dummies (Numéricas para el modelo)
# drop_first=True es vital para evitar la trampa de la variable ficticia
```

```

df_final_model = pd.get_dummies(df_reducido, drop_first=True)

# 4. Definir X e y
y = df_final_model['Exam']
X = df_final_model.drop(columns=['Exam']).astype(float)

# 5. Añadir constante
X = sm.add_constant(X)

# 6. Ajustar modelo final
modelo_final = sm.OLS(y, X).fit()

# 7. Resultados
print(modelo_final.summary())

```

OLS Regression Results

=====					
Dep. Variable:		Exam	R-squared:		0.266
Model:		OLS	Adj. R-squared:		0.263
Method:		Least Squares	F-statistic:		124.8
Date:		mié., 22 abr. 2026	Prob (F-statistic):		0.00
Time:		10:12:25	Log-Likelihood:		-1214.1
No. Observations:		5193	AIC:		2460.
Df Residuals:		5177	BIC:		2565.
Df Model:		15			
Covariance Type:		nonrobust			
=====					
=====					
		coef	std err	t	P> t

[0.025 0.975]					

const		0.4503	0.032	14.272	0.000
0.388	0.512				
focus_index		0.0046	0.001	6.660	0.000
0.003	0.006				
burnout_level		-0.0023	0.000	-6.621	0.000
-0.003	-0.002				
productivity_score		0.0055	0.000	13.001	0.000
0.005	0.006				
gaming_hours_Index		0.0213	0.012	1.751	0.080
-0.003	0.045				
online_classes_hours_index		0.1167	0.017	6.674	0.000
0.082	0.151				
exercise_hours_index		0.0476	0.012	3.962	0.000
0.024	0.071				
gender_Male		0.0077	0.009	0.824	0.410
-0.011	0.026				

gender_Other	-0.0342	0.013	-2.692	0.007
-0.059	-0.009			
academic_level_Other	-0.2640	0.024	-10.964	0.000
-0.311	-0.217			
academic_level_Postgraduate	0.0108	0.011	1.019	0.308
-0.010	0.032			
academic_level_Undergraduate	0.0329	0.011	3.063	0.002
0.012	0.054			
part_time_job_Other	-0.1928	0.020	-9.568	0.000
-0.232	-0.153			
part_time_job_Yes	-0.0025	0.009	-0.260	0.795
-0.021	0.016			
internet_quality_Good	0.0240	0.010	2.318	0.020
0.004	0.044			
internet_quality_Poor	0.0117	0.010	1.116	0.264
-0.009	0.032			

Omnibus:	1096.296	Durbin-Watson:	2.028
Prob(Omnibus):	0.000	Jarque-Bera (JB):	2014.061
Skew:	-1.324	Prob(JB):	0.00
Kurtosis:	4.515	Cond. No.	556.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[7]: #PREGUNTA 2.3
#MODELO MCO CON VARIABLES DE INTERES SIN CORRELACION (entre variables_
    ↳ predictoras )
import statsmodels.api as sm
import pandas as pd

# 1. Selección de variables solicitadas
# Nota: Usamos df_limpio que ya tiene las transformaciones y limpieza final
vars_modelo = [
    'gaming_hours',
    'exercise_hours',
    'mental_health_score',
    'burnout_level',
    'productivity_score',
    'online_classes_hours_index'
]

# 2. Preparación de matrices
# Aseguramos que los datos sean numéricos y manejables por el algoritmo
X = df_limpio[vars_modelo].astype(float)
```

```

y = df_limpio['Exam']

# 3. Añadir la constante (Intercepto)
# Indispensable en MCO para que los residuos tengan media cero
X = sm.add_constant(X)

# 4. Ajuste del modelo MCO (OLS)
modelo_conductual = sm.OLS(y, X).fit()

# 5. Visualización de resultados
print("="*60)
print("MODELO MCO: DETERMINANTES CONDUCTUALES DE LA ASISTENCIA")
print("="*60)
print(modelo_conductual.summary())

# 6. Guardar coeficientes para análisis rápido
coeficientes = modelo_conductual.params
print("\nInterpretación rápida de coeficientes:")
print(coeficientes)

```

```

=====
MODELO MCO: DETERMINANTES CONDUCTUALES DE LA ASISTENCIA
=====

                        OLS Regression Results
=====
Dep. Variable:          Exam      R-squared:                0.185
Model:                  OLS      Adj. R-squared:            0.185
Method:                 Least Squares  F-statistic:           196.8
Date:                  mié., 22 abr. 2026  Prob (F-statistic):    1.17e-226
Time:                  10:12:55    Log-Likelihood:        -1482.7
No. Observations:      5193      AIC:                  2979.
Df Residuals:          5186      BIC:                  3025.
Df Model:               6
Covariance Type:       nonrobust
=====
=====

```

	coef	std err	t	P> t
[0.025 0.975]				

const	0.4058	0.029	13.768	0.000
0.348 0.464				
gaming_hours	-0.0070	0.004	-1.684	0.092
-0.015 0.001				
exercise_hours	0.0005	0.007	0.076	0.939
-0.013 0.014				
mental_health_score	0.0021	0.002	1.009	0.313
-0.002 0.006				

burnout_level	-0.0019	0.000	-5.576	0.000
-0.003	-0.001			
productivity_score	0.0074	0.000	20.361	0.000
0.007	0.008			
online_classes_hours_index	0.2750	0.017	16.304	0.000
0.242	0.308			

Omnibus:	1192.212	Durbin-Watson:	2.024
Prob(Omnibus):	0.000	Jarque-Bera (JB):	2217.863
Skew:	-1.452	Prob(JB):	0.00
Kurtosis:	4.348	Cond. No.	425.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Interpretación rápida de coeficientes:

const	0.405826
gaming_hours	-0.006962
exercise_hours	0.000521
mental_health_score	0.002093
burnout_level	-0.001935
productivity_score	0.007436
online_classes_hours_index	0.275046

dtype: float64

```
[8]: #PREGUNTA 3.1
#modelo provit
import statsmodels.api as sm
import pandas as pd

# 1. Preparación de datos (Dummies y limpieza de nombres)
df_probit = pd.get_dummies(df_limpio, drop_first=True)
X = df_probit.drop(columns=['Exam', 'exam_score', 'student_id'],
    ↪errors='ignore')
y = df_probit['Exam']
X = sm.add_constant(X).astype(float)

# 2. AJUSTE DEL MODELO PROBIT (Summary estándar)
# Este paso es necesario para ver el ajuste global del modelo
modelo_probit = sm.Probit(y, X).fit()

print("\n" + "="*80)
print("1. RESUMEN GLOBAL DEL MODELO PROBIT (Coeficientes Beta)")
print("="*80)
print(modelo_probit.summary())
```

```
# 3. CÁLCULO DE EFECTOS MARGINALES (dy/dx)
# Calculamos el Efecto Marginal Promedio (AME)
probit_margeff = modelo_probit.get_margeff(at='overall', method='dydx')

print("\n" + "="*80)
print("2. EFECTOS MARGINALES DEL MODELO PROBIT (Interpretación dy/dx)")
print("="*80)
print(probit_margeff.summary())
```

Optimization terminated successfully.
 Current function value: 0.282062
 Iterations 7

1. RESUMEN GLOBAL DEL MODELO PROBIT (Coeficientes Beta)

Probit Regression Results

Dep. Variable:	Exam	No. Observations:	5193
Model:	Probit	Df Residuals:	5165
Method:	MLE	Df Model:	27
Date:	mié., 22 abr. 2026	Pseudo R-squ.:	0.3317
Time:	10:14:09	Log-Likelihood:	-1464.7
converged:	True	LL-Null:	-2191.7
Covariance Type:	nonrobust	LLR p-value:	6.504e-290

		coef	std err	z	P> z
[0.025 0.975]					

const		-0.9992	0.399	-2.507	0.012
-1.780	-0.218				
age		-0.0047	0.009	-0.502	0.615
-0.023	0.014				
study_hours		0.0078	0.026	0.294	0.769
-0.044	0.059				
self_study_hours		-0.0089	0.025	-0.357	0.721
-0.058	0.040				
online_classes_hours		-0.0013	0.028	-0.048	0.962
-0.056	0.054				
social_media_hours		-0.0019	0.021	-0.092	0.927
-0.043	0.039				
gaming_hours		-0.0287	0.027	-1.050	0.294
-0.082	0.025				
sleep_hours		0.0393	0.032	1.231	0.218
-0.023	0.102				

screen_time_hours	-0.0138	0.013	-1.064	0.287
-0.039 0.012				
caffeine_intake_mg	-0.0002	0.000	-1.128	0.259
-0.001 0.000				
upcoming_deadline	0.0120	0.076	0.158	0.874
-0.137 0.161				
mental_health_score	0.0089	0.017	0.511	0.609
-0.025 0.043				
focus_index	0.0256	0.005	5.008	0.000
0.016 0.036				
burnout_level	-0.0116	0.004	-3.045	0.002
-0.019 -0.004				
productivity_score	0.0358	0.004	8.632	0.000
0.028 0.044				
gaming_hours_Index	0.1698	0.077	2.209	0.027
0.019 0.321				
online_classes_hours_index	0.5269	0.090	5.882	0.000
0.351 0.702				
exercise_hours	0.0263	0.042	0.630	0.529
-0.056 0.108				
exercise_hours_index	0.2347	0.069	3.386	0.001
0.099 0.371				
gender_Male	0.0196	0.058	0.338	0.736
-0.094 0.134				
gender_Other	-0.1912	0.073	-2.602	0.009
-0.335 -0.047				
academic_level_Other	-0.8903	0.114	-7.778	0.000
-1.115 -0.666				
academic_level_Postgraduate	0.0714	0.065	1.100	0.271
-0.056 0.199				
academic_level_Undergraduate	0.2051	0.067	3.040	0.002
0.073 0.337				
part_time_job_Other	-0.8192	0.099	-8.290	0.000
-1.013 -0.625				
part_time_job_Yes	-0.0255	0.066	-0.384	0.701
-0.156 0.105				
internet_quality_Good	0.1354	0.064	2.126	0.034
0.011 0.260				
internet_quality_Poor	0.0720	0.064	1.134	0.257
-0.052 0.196				

=====

=====

2. EFECTOS MARGINALES DEL MODELO PROBIT (Interpretación dy/dx)

=====

Probit Marginal Effects

=====

Dep. Variable:

Exam

Method:

dydx

At:

overall

[0.025 0.975]		dy/dx	std err	z	P> z

age		-0.0008	0.002	-0.503	0.615
-0.004	0.002				
study_hours		0.0012	0.004	0.294	0.769
-0.007	0.010				
self_study_hours		-0.0014	0.004	-0.357	0.721
-0.009	0.006				
online_classes_hours		-0.0002	0.004	-0.048	0.962
-0.009	0.009				
social_media_hours		-0.0003	0.003	-0.092	0.927
-0.007	0.006				
gaming_hours		-0.0046	0.004	-1.051	0.293
-0.013	0.004				
sleep_hours		0.0063	0.005	1.231	0.218
-0.004	0.016				
screen_time_hours		-0.0022	0.002	-1.064	0.287
-0.006	0.002				
caffeine_intake_mg		-3.64e-05	3.22e-05	-1.129	0.259
-9.96e-05	2.68e-05				
upcoming_deadline		0.0019	0.012	0.158	0.874
-0.022	0.026				
mental_health_score		0.0014	0.003	0.511	0.609
-0.004	0.007				
focus_index		0.0041	0.001	5.029	0.000
0.002	0.006				
burnout_level		-0.0019	0.001	-3.048	0.002
-0.003	-0.001				
productivity_score		0.0057	0.001	8.729	0.000
0.004	0.007				
gaming_hours_Index		0.0272	0.012	2.212	0.027
0.003	0.051				
online_classes_hours_index		0.0844	0.014	5.938	0.000
0.057	0.112				
exercise_hours		0.0042	0.007	0.630	0.529
-0.009	0.017				
exercise_hours_index		0.0376	0.011	3.398	0.001
0.016	0.059				
gender_Male		0.0031	0.009	0.338	0.736
-0.015	0.021				
gender_Other		-0.0306	0.012	-2.603	0.009

-0.054	-0.008				
academic_level_Other		-0.1426	0.018	-7.896	0.000
-0.178	-0.107				
academic_level_Postgraduate		0.0114	0.010	1.100	0.271
-0.009	0.032				
academic_level_Undergraduate		0.0328	0.011	3.047	0.002
0.012	0.054				
part_time_job_Other		-0.1312	0.016	-8.445	0.000
-0.162	-0.101				
part_time_job_Yes		-0.0041	0.011	-0.384	0.701
-0.025	0.017				
internet_quality_Good		0.0217	0.010	2.129	0.033
0.002	0.042				
internet_quality_Poor		0.0115	0.010	1.134	0.257
-0.008	0.031				

=====

=====

```
[9]: #PREGUNTA 3.2
#modelo probit significativo
import statsmodels.api as sm
import pandas as pd

# 1. Lista de variables originales que contienen categorías significativas
vars_originales = [
    'focus_index',
    'burnout_level',
    'productivity_score',
    'gaming_hours_Index',
    'online_classes_hours_index',
    'exercise_hours_index',
    'gender',          # Contiene 'Other'
    'academic_level',  # Contiene 'Undergraduate' y 'Other'
    'part_time_job',   # Contiene 'Other'
    'internet_quality' # Contiene 'Good'
]

# 2. Preparación del DataFrame
# Seleccionamos las columnas y creamos dummies automáticamente
df_model = pd.get_dummies(df_limpio[vars_originales + ['Exam']],
    drop_first=True)

# 3. Definir X e y
y = df_model['Exam']
X = df_model.drop(columns=['Exam']).astype(float)
X = sm.add_constant(X)
```

```

# 4. Ajuste del Modelo Probit
modelo_probit_final = sm.Probit(y, X).fit()

# 5. RESULTADOS: Summary y Efectos Marginales
print("\n" + "="*80)
print("1. RESUMEN GLOBAL (COEFICIENTES BETA)")
print("="*80)
print(modelo_probit_final.summary())

# Cálculo de dy/dx
margeff_final = modelo_probit_final.get_margeff(at='overall', method='dydx')

print("\n" + "="*80)
print("2. EFECTOS MARGINALES (INTERPRETACIÓN dy/dx)")
print("="*80)
print(margeff_final.summary())

```

Optimization terminated successfully.
Current function value: 0.282605
Iterations 7

1. RESUMEN GLOBAL (COEFICIENTES BETA)

Probit Regression Results					
=====					
Dep. Variable:		Exam	No. Observations:		5193
Model:		Probit	Df Residuals:		5177
Method:		MLE	Df Model:		15
Date:	mié., 22 abr. 2026	Pseudo R-squ.:		0.3304	
Time:	10:15:45	Log-Likelihood:		-1467.6	
converged:		True	LL-Null:		-2191.7
Covariance Type:		nonrobust	LLR p-value:		6.551e-300
=====					
=====					
		coef	std err	z	P> z
[0.025 0.975]					

const		-0.8755	0.181	-4.849	0.000
-1.229	-0.522				
focus_index		0.0274	0.004	6.567	0.000
0.019	0.036				
burnout_level		-0.0144	0.002	-6.777	0.000
-0.019	-0.010				
productivity_score		0.0364	0.003	13.594	0.000
0.031	0.042				
gaming_hours Index		0.1414	0.070	2.021	0.043

0.004	0.279				
online_classes_hours_index		0.5271	0.088	5.976	0.000
0.354	0.700				
exercise_hours_index		0.2366	0.069	3.433	0.001
0.102	0.372				
gender_Male		0.0213	0.058	0.368	0.713
-0.092	0.135				
gender_Other		-0.1868	0.073	-2.548	0.011
-0.330	-0.043				
academic_level_Other		-0.8989	0.114	-7.906	0.000
-1.122	-0.676				
academic_level_Postgraduate		0.0716	0.065	1.106	0.269
-0.055	0.199				
academic_level_Undergraduate		0.2029	0.067	3.018	0.003
0.071	0.335				
part_time_job_Other		-0.7992	0.097	-8.242	0.000
-0.989	-0.609				
part_time_job_Yes		0.0053	0.059	0.089	0.929
-0.110	0.121				
internet_quality_Good		0.1398	0.063	2.203	0.028
0.015	0.264				
internet_quality_Poor		0.0697	0.063	1.101	0.271
-0.054	0.194				

=====

=====

2. EFECTOS MARGINALES (INTERPRETACIÓN dy/dx)

Probit Marginal Effects

Dep. Variable: Exam

Method: dydx

At: overall

		dy/dx	std err	z	P> z
[0.025	0.975]				
focus_index		0.0044	0.001	6.619	0.000
0.003	0.006				
burnout_level		-0.0023	0.000	-6.826	0.000
-0.003	-0.002				
productivity_score		0.0058	0.000	13.939	0.000
0.005	0.007				
gaming_hours_Index		0.0227	0.011	2.023	0.043
0.001	0.045				

online_classes_hours_index	0.0846	0.014	6.035	0.000
0.057	0.112			
exercise_hours_index	0.0380	0.011	3.446	0.001
0.016	0.060			
gender_Male	0.0034	0.009	0.368	0.713
-0.015	0.022			
gender_Other	-0.0300	0.012	-2.550	0.011
-0.053	-0.007			
academic_level_Other	-0.1442	0.018	-8.030	0.000
-0.179	-0.109			
academic_level_Postgraduate	0.0115	0.010	1.106	0.269
-0.009	0.032			
academic_level_Undergraduate	0.0326	0.011	3.024	0.002
0.011	0.054			
part_time_job_Other	-0.1282	0.015	-8.392	0.000
-0.158	-0.098			
part_time_job_Yes	0.0008	0.009	0.089	0.929
-0.018	0.019			
internet_quality_Good	0.0224	0.010	2.206	0.027
0.003	0.042			
internet_quality_Poor	0.0112	0.010	1.101	0.271
-0.009	0.031			

=====

=====

```
[10]: #PREGUNTA 4.1
#modelo logit completo
import statsmodels.api as sm
import pandas as pd
import numpy as np

# 1. Preparación de los datos (Dummies automáticas)
df_logit = pd.get_dummies(df_limpio, drop_first=True)

# 2. Definición de variables (Variables originales significativas según tus
    pruebas)
# Eliminamos los IDs y la nota del examen para evitar sesgos
X = df_logit.drop(columns=['Exam', 'exam_score', 'student_id'],
    errors='ignore').astype(float)
y = df_logit['Exam']

# 3. Añadir constante
X = sm.add_constant(X)

# 4. Ajuste del modelo Logit
modelo_logit = sm.Logit(y, X).fit()
```

```

# 5. RESULTADOS: Resumen Global y Coeficientes Log-Odds
print("\n" + "="*80)
print("1. RESUMEN GLOBAL DEL MODELO LOGIT")
print("="*80)
print(modelo_logit.summary())

# 6. RESULTADOS: Odds Ratios (Interpretación de ventajas)
print("\n" + "="*80)
print("2. ODDS RATIOS (Interpretación de Probabilidades Relativas)")
print("="*80)
odds_ratios = np.exp(modelo_logit.params)
print(odds_ratios)

# 7. RESULTADOS: Efectos Marginales (dy/dx para probabilidad directa)
print("\n" + "="*80)
print("3. EFECTOS MARGINALES (Cambio en probabilidad porcentual)")
print("="*80)
margeff_logit = modelo_logit.get_margeff(at='overall', method='dydx')
print(margeff_logit.summary())

```

Optimization terminated successfully.
Current function value: 0.275469
Iterations 8

1. RESUMEN GLOBAL DEL MODELO LOGIT

Logit Regression Results

Dep. Variable:	Exam	No. Observations:	5193
Model:	Logit	Df Residuals:	5165
Method:	MLE	Df Model:	27
Date:	mié., 22 abr. 2026	Pseudo R-squ.:	0.3473
Time:	10:16:42	Log-Likelihood:	-1430.5
converged:	True	LL-Null:	-2191.7
Covariance Type:	nonrobust	LLR p-value:	1.564e-304

		coef	std err	z	P> z

const		-2.0161	0.743	-2.712	0.007
-3.473	-0.559				
age		-0.0048	0.017	-0.277	0.782
-0.039	0.029				
study_hours		0.0174	0.050	0.346	0.729
-0.081	0.116				

self_study_hours	-0.0261	0.046	-0.566	0.571
-0.116 0.064				
online_classes_hours	0.0005	0.053	0.010	0.992
-0.103 0.104				
social_media_hours	-0.0007	0.039	-0.019	0.985
-0.076 0.075				
gaming_hours	-0.0423	0.050	-0.842	0.400
-0.141 0.056				
sleep_hours	0.0705	0.060	1.179	0.239
-0.047 0.188				
screen_time_hours	-0.0245	0.024	-1.022	0.307
-0.072 0.023				
caffeine_intake_mg	-0.0003	0.000	-0.862	0.389
-0.001 0.000				
upcoming_deadline	0.0307	0.142	0.217	0.829
-0.247 0.308				
mental_health_score	0.0194	0.033	0.587	0.557
-0.045 0.084				
focus_index	0.0519	0.010	5.389	0.000
0.033 0.071				
burnout_level	-0.0251	0.007	-3.517	0.000
-0.039 -0.011				
productivity_score	0.0736	0.008	9.018	0.000
0.058 0.090				
gaming_hours_Index	0.2958	0.143	2.076	0.038
0.016 0.575				
online_classes_hours_index	0.9631	0.163	5.914	0.000
0.644 1.282				
exercise_hours	0.0151	0.077	0.195	0.845
-0.136 0.167				
exercise_hours_index	0.4003	0.128	3.138	0.002
0.150 0.650				
gender_Male	-0.0016	0.108	-0.015	0.988
-0.214 0.211				
gender_Other	-0.4242	0.134	-3.159	0.002
-0.687 -0.161				
academic_level_Other	-1.7132	0.205	-8.340	0.000
-2.116 -1.311				
academic_level_Postgraduate	0.1291	0.120	1.075	0.283
-0.106 0.364				
academic_level_Undergraduate	0.3812	0.126	3.024	0.002
0.134 0.628				
part_time_job_Other	-1.5119	0.182	-8.308	0.000
-1.869 -1.155				
part_time_job_Yes	-0.0658	0.124	-0.531	0.595
-0.309 0.177				
internet_quality_Good	0.2281	0.118	1.938	0.053
-0.003 0.459				

internet_quality_Poor	0.1434	0.118	1.217	0.224
-0.088	0.374			

2. ODDS RATIOS (Interpretación de Probabilidades Relativas)

const	0.133170
age	0.995177
study_hours	1.017584
self_study_hours	0.974247
online_classes_hours	1.000508
social_media_hours	0.999253
gaming_hours	0.958545
sleep_hours	1.073045
screen_time_hours	0.975756
caffeine_intake_mg	0.999679
upcoming_deadline	1.031153
mental_health_score	1.019550
focus_index	1.053292
burnout_level	0.975187
productivity_score	1.076417
gaming_hours_Index	1.344203
online_classes_hours_index	2.619863
exercise_hours	1.015209
exercise_hours_index	1.492203
gender_Male	0.998361
gender_Other	0.654283
academic_level_Other	0.180293
academic_level_Postgraduate	1.137766
academic_level_Undergraduate	1.464011
part_time_job_Other	0.220494
part_time_job_Yes	0.936274
internet_quality_Good	1.256209
internet_quality_Poor	1.154232

dtype: float64

3. EFECTOS MARGINALES (Cambio en probabilidad porcentual)

Logit Marginal Effects

Dep. Variable:	Exam
Method:	dydx
At:	overall

[0.025 0.975]		dy/dx	std err	z	P> z

age		-0.0004	0.001	-0.277	0.782
-0.003	0.002				
study_hours		0.0015	0.004	0.346	0.729
-0.007	0.010				
self_study_hours		-0.0022	0.004	-0.566	0.571
-0.010	0.005				
online_classes_hours		4.255e-05	0.004	0.010	0.992
-0.009	0.009				
social_media_hours		-6.259e-05	0.003	-0.019	0.985
-0.006	0.006				
gaming_hours		-0.0035	0.004	-0.843	0.399
-0.012	0.005				
sleep_hours		0.0059	0.005	1.179	0.238
-0.004	0.016				
screen_time_hours		-0.0021	0.002	-1.022	0.307
-0.006	0.002				
caffeine_intake_mg		-2.687e-05	3.12e-05	-0.862	0.389
-8.8e-05	3.42e-05				
upcoming_deadline		0.0026	0.012	0.217	0.829
-0.021	0.026				
mental_health_score		0.0016	0.003	0.587	0.557
-0.004	0.007				
focus_index		0.0044	0.001	5.437	0.000
0.003	0.006				
burnout_level		-0.0021	0.001	-3.530	0.000
-0.003	-0.001				
productivity_score		0.0062	0.001	9.237	0.000
0.005	0.007				
gaming_hours_Index		0.0248	0.012	2.079	0.038
0.001	0.048				
online_classes_hours_index		0.0807	0.013	5.993	0.000
0.054	0.107				
exercise_hours		0.0013	0.006	0.195	0.845
-0.011	0.014				
exercise_hours_index		0.0335	0.011	3.151	0.002
0.013	0.054				
gender_Male		-0.0001	0.009	-0.015	0.988
-0.018	0.018				
gender_Other		-0.0355	0.011	-3.169	0.002
-0.058	-0.014				
academic_level_Other		-0.1436	0.017	-8.570	0.000
-0.176	-0.111				
academic_level_Postgraduate		0.0108	0.010	1.075	0.282
-0.009	0.031				

academic_level_Undergraduate	0.0319	0.011	3.034	0.002
0.011	0.053			
part_time_job_Other	-0.1267	0.015	-8.515	0.000
-0.156	-0.098			
part_time_job_Yes	-0.0055	0.010	-0.531	0.595
-0.026	0.015			
internet_quality_Good	0.0191	0.010	1.940	0.052
-0.000	0.038			
internet_quality_Poor	0.0120	0.010	1.217	0.223
-0.007	0.031			

=====

=====

```
[11]: #PREGUNTA 4.2
#Modelo logit significativo
import statsmodels.api as sm
import pandas as pd
import numpy as np

# 1. Definimos las variables originales que contienen las categorías
↳significativas
vars_originales_logit = [
    'focus_index',
    'burnout_level',
    'productivity_score',
    'gaming_hours_Index',
    'online_classes_hours_index',
    'exercise_hours_index',
    'gender',          # Contiene gender_Other
    'academic_level',  # Contiene academic_level_Other y _Undergraduate
    'part_time_job',   # Contiene part_time_job_Other
    'internet_quality' # Contiene internet_quality_Good
]

# 2. Preparación del DataFrame y creación de dummies
df_model_logit = pd.get_dummies(df_limpio[vars_originales_logit + ['Exam']],
↳drop_first=True)

# 3. Definir X e y (asegurando tipo float para los cálculos del Logit)
y = df_model_logit['Exam']
X = df_model_logit.drop(columns=['Exam']).astype(float)
X = sm.add_constant(X)

# 4. Ajuste del Modelo Logit
modelo_logit_final = sm.Logit(y, X).fit()

# 5. RESULTADOS: Resumen y Estadísticos de Ajuste
```

```

print("\n" + "="*80)
print("1. RESUMEN GLOBAL DEL MODELO LOGIT (Coeficientes Beta)")
print("="*80)
print(modelo_logit_final.summary())

# 6. RESULTADOS: Odds Ratios (Interpretación de Ventaja Relativa)
print("\n" + "="*80)
print("2. ODDS RATIOS (Exp(Beta))")
print("="*80)
print(np.exp(modelo_logit_final.params))

# 7. RESULTADOS: Efectos Marginales (dy/dx para probabilidad directa)
margeff_logit_final = modelo_logit_final.get_margeff(at='overall',
↳method='dydx')
print("\n" + "="*80)
print("3. EFECTOS MARGINALES DEL MODELO LOGIT (Interpretación dy/dx)")
print("="*80)
print(margeff_logit_final.summary())

```

Optimization terminated successfully.

Current function value: 0.275929

Iterations 8

1. RESUMEN GLOBAL DEL MODELO LOGIT (Coeficientes Beta)

Logit Regression Results

```

=====
Dep. Variable:          Exam    No. Observations:          5193
Model:                Logit    Df Residuals:              5177
Method:                MLE     Df Model:                15
Date:                 mié., 22 abr. 2026    Pseudo R-squ.:          0.3462
Time:                 10:17:54    Log-Likelihood:         -1432.9
converged:              True     LL-Null:                -2191.7
Covariance Type:       nonrobust    LLR p-value:            0.000
=====

```

```

=====
                                coef    std err          z      P>|z|
-----
[0.025    0.975]
-----
const                -1.7540      0.328     -5.349      0.000
-2.397    -1.111
focus_index          0.0542      0.008      6.865      0.000
0.039     0.070
burnout_level        -0.0291      0.004     -7.358      0.000
-0.037    -0.021
productivity_score     0.0758      0.005     14.367      0.000

```

0.065	0.086				
gaming_hours_Index		0.2513	0.129	1.946	0.052
-0.002	0.504				
online_classes_hours_index		0.9650	0.160	6.037	0.000
0.652	1.278				
exercise_hours_index		0.4026	0.127	3.178	0.001
0.154	0.651				
gender_Male		-0.0024	0.108	-0.022	0.983
-0.215	0.210				
gender_Other		-0.4176	0.134	-3.116	0.002
-0.680	-0.155				
academic_level_Other		-1.7260	0.204	-8.465	0.000
-2.126	-1.326				
academic_level_Postgraduate		0.1292	0.120	1.078	0.281
-0.106	0.364				
academic_level_Undergraduate		0.3779	0.126	3.010	0.003
0.132	0.624				
part_time_job_Other		-1.4840	0.178	-8.336	0.000
-1.833	-1.135				
part_time_job_Yes		-0.0173	0.109	-0.159	0.874
-0.232	0.197				
internet_quality_Good		0.2375	0.117	2.027	0.043
0.008	0.467				
internet_quality_Poor		0.1401	0.118	1.192	0.233
-0.090	0.371				

=====

=====

=====

2. ODDS RATIOS (Exp(Beta))

=====

const	0.173076
focus_index	1.055742
burnout_level	0.971360
productivity_score	1.078776
gaming_hours_Index	1.285717
online_classes_hours_index	2.624723
exercise_hours_index	1.495654
gender_Male	0.997647
gender_Other	0.658647
academic_level_Other	0.177989
academic_level_Postgraduate	1.137951
academic_level_Undergraduate	1.459175
part_time_job_Other	0.226737
part_time_job_Yes	0.982809
internet_quality_Good	1.268125
internet_quality_Poor	1.150439

dtype: float64

=====

3. EFECTOS MARGINALES DEL MODELO LOGIT (Interpretación dy/dx)

=====

Logit Marginal Effects

=====

Dep. Variable: Exam
Method: dydx
At: overall

=====

	dy/dx	std err	z	P> z
[0.025 0.975]				

focus_index	0.0046	0.001	6.969	0.000
0.003 0.006				
burnout_level	-0.0024	0.000	-7.481	0.000
-0.003 -0.002				
productivity_score	0.0064	0.000	15.309	0.000
0.006 0.007				
gaming_hours_Index	0.0211	0.011	1.949	0.051
-0.000 0.042				
online_classes_hours_index	0.0810	0.013	6.120	0.000
0.055 0.107				
exercise_hours_index	0.0338	0.011	3.191	0.001
0.013 0.055				
gender_Male	-0.0002	0.009	-0.022	0.983
-0.018 0.018				
gender_Other	-0.0350	0.011	-3.126	0.002
-0.057 -0.013				
academic_level_Other	-0.1449	0.017	-8.707	0.000
-0.177 -0.112				
academic_level_Postgraduate	0.0108	0.010	1.079	0.281
-0.009 0.031				
academic_level_Undergraduate	0.0317	0.011	3.020	0.003
0.011 0.052				
part_time_job_Other	-0.1245	0.015	-8.544	0.000
-0.153 -0.096				
part_time_job_Yes	-0.0015	0.009	-0.159	0.874
-0.019 0.017				
internet_quality_Good	0.0199	0.010	2.030	0.042
0.001 0.039				
internet_quality_Poor	0.0118	0.010	1.193	0.233
-0.008 0.031				

=====

=====

```
[12]: #PREGUNTA 6.1
#MODELO POISSON (6) TODAS LAS VARIABLES
import statsmodels.api as sm
import pandas as pd
import numpy as np

# 1. Filtrado: Solo alumnos que rindieron el examen
# Es fundamental modelar el desempeño solo sobre la población observada
df_asistentes = df_limpio[df_limpio['Exam'] == 1].copy()

# 2. Preparación de variables (Creación de dummies para todas las categóricas)
# Excluimos 'Exam' porque ya es constante (1) y 'student_id'
df_poisson_full = pd.get_dummies(df_asistentes.drop(columns=['student_id', 'Exam'], errors='ignore'), drop_first=True)

# 3. Definir variable dependiente e independientes
y = df_poisson_full['exam_score']
X = df_poisson_full.drop(columns=['exam_score']).astype(float)
X = sm.add_constant(X)

# 4. Ajuste del Modelo de Poisson
# Usamos el Modelo Lineal Generalizado (GLM) con familia Poisson
modelo_poisson_full = sm.GLM(y, X, family=sm.families.Poisson()).fit()

# 5. Resultados
print("\n" + "="*80)
print("RESUMEN DEL MODELO POISSON COMPLETO (Determinantes de la Nota)")
print("="*80)
print(modelo_poisson_full.summary())

# 6. Cálculo de los Incidence Rate Ratios (IRR)
# El IRR es vital para la interpretación en Poisson
irr_full = np.exp(modelo_poisson_full.params)
print("\n" + "="*40)
print("INCIDENCE RATE RATIOS (IRR)")
print("="*40)
print(irr_full)
```

```
=====
RESUMEN DEL MODELO POISSON COMPLETO (Determinantes de la Nota)
=====
```

Generalized Linear Model Regression Results

```
=====
Dep. Variable:          exam_score    No. Observations:          4416
Model:                  GLM          Df Residuals:              4388
Model Family:           Poisson      Df Model:                  27
Link Function:          Log          Scale:                     1.0000
```

Method: IRLS Log-Likelihood: -14312.
Date: mié., 22 abr. 2026 Deviance: 7993.9
Time: 10:18:14 Pearson chi2: 7.40e+03
No. Iterations: 5 Pseudo R-squ. (CS): 0.9913
Covariance Type: nonrobust

		coef	std err	z	P> z
[0.025 0.975]					

const		1.9782	0.053	37.025	0.000
1.873	2.083				
age		0.0006	0.001	0.522	0.602
-0.002	0.003				
study_hours		0.0169	0.004	4.524	0.000
0.010	0.024				
self_study_hours		-0.0043	0.003	-1.393	0.164
-0.010	0.002				
online_classes_hours		-0.0021	0.003	-0.589	0.556
-0.009	0.005				
social_media_hours		0.0024	0.003	0.909	0.364
-0.003	0.008				
gaming_hours		0.0038	0.004	1.083	0.279
-0.003	0.011				
sleep_hours		0.0048	0.004	1.122	0.262
-0.004	0.013				
screen_time_hours		-0.0024	0.002	-1.455	0.146
-0.006	0.001				
caffeine_intake_mg		7.793e-07	2.56e-05	0.030	0.976
-4.93e-05	5.09e-05				
upcoming_deadline		-0.0364	0.010	-3.671	0.000
-0.056	-0.017				
mental_health_score		0.0165	0.002	6.861	0.000
0.012	0.021				
focus_index		0.0143	0.001	20.642	0.000
0.013	0.016				
burnout_level		-0.0088	0.001	-16.295	0.000
-0.010	-0.008				
productivity_score		0.0185	0.001	31.078	0.000
0.017	0.020				
gaming_hours_Index		-0.0105	0.011	-0.975	0.330
-0.032	0.011				
online_classes_hours_index		-0.0382	0.015	-2.509	0.012
-0.068	-0.008				
exercise_hours		-9.564e-05	0.005	-0.018	0.985
-0.010	0.010				
exercise_hours_index		0.0140	0.009	1.473	0.141

-0.005	0.033				
gender_Male		0.0012	0.007	0.163	0.870
-0.013	0.015				
gender_Other		0.0062	0.010	0.606	0.544
-0.014	0.026				
academic_level_Other		0.0684	0.024	2.882	0.004
0.022	0.115				
academic_level_Postgraduate		-0.0056	0.008	-0.677	0.499
-0.022	0.011				
academic_level_Undergraduate		0.0019	0.008	0.230	0.818
-0.014	0.018				
part_time_job_Other		0.0356	0.019	1.901	0.057
-0.001	0.072				
part_time_job_Yes		-0.0125	0.009	-1.451	0.147
-0.029	0.004				
internet_quality_Good		0.0057	0.008	0.709	0.479
-0.010	0.022				
internet_quality_Poor		-0.0013	0.008	-0.164	0.870
-0.017	0.015				

=====

=====

=====

INCIDENCE RATE RATIOS (IRR)

=====

const	7.229780
age	1.000616
study_hours	1.017062
self_study_hours	0.995692
online_classes_hours	0.997951
social_media_hours	1.002381
gaming_hours	1.003822
sleep_hours	1.004820
screen_time_hours	0.997570
caffeine_intake_mg	1.000001
upcoming_deadline	0.964229
mental_health_score	1.016607
focus_index	1.014415
burnout_level	0.991285
productivity_score	1.018659
gaming_hours_Index	0.989540
online_classes_hours_index	0.962526
exercise_hours	0.999904
exercise_hours_index	1.014078
gender_Male	1.001176
gender_Other	1.006205
academic_level_Other	1.070839
academic_level_Postgraduate	0.994448

```

academic_level_Undergraduate    1.001906
part_time_job_Other              1.036286
part_time_job_Yes                0.987568
internet_quality_Good            1.005749
internet_quality_Poor            0.998665
dtype: float64

```

```

[33]: # PREGUNTA 6.1: MODELO POISSON (6) CON CÁLCULO DE PSEUDO R2
import statsmodels.api as sm
import pandas as pd
import numpy as np

# 1. Filtrado: Solo alumnos que rindieron el examen
df_asistentes = df_limpio[df_limpio['Exam'] == 1].copy()

# 2. Preparación de variables
df_poisson_full = pd.get_dummies(df_asistentes.drop(columns=['student_id', 'Exam'], errors='ignore'), drop_first=True)

# 3. Definir variable dependiente e independientes
y = df_poisson_full['exam_score']
X = df_poisson_full.drop(columns=['exam_score']).astype(float)
X = sm.add_constant(X)

# 4. Ajuste del Modelo de Poisson y Modelo Nulo
modelo_poisson_full = sm.GLM(y, X, family=sm.families.Poisson()).fit()

# Ajuste del modelo nulo (solo intercepto) para calcular Pseudo R2
modelo_nulo = sm.GLM(y, np.ones((len(y), 1)), family=sm.families.Poisson()).fit()

# 5. Cálculo de Pseudo R-cuadrados
logloss_full = modelo_poisson_full.llf
logloss_null = modelo_nulo.llf
n = len(y)

# McFadden
pseudo_r2_mcfadden = 1 - (logloss_full / logloss_null)

# Cox & Snell
pseudo_r2_cox_snell = 1 - np.exp((2 / n) * (logloss_null - logloss_full))

# 6. Resultados
print("\n" + "="*80)
print("RESUMEN DEL MODELO POISSON COMPLETO (Determinantes de la Nota)")
print("="*80)
print(modelo_poisson_full.summary())

```

```

print("\n" + "="*40)
print("MÉTRICAS DE AJUSTE (PSEUDO R2)")
print("="*40)
print(f"Pseudo R2 (McFadden): {pseudo_r2_mcfadden:.4f}")
print(f"Pseudo R2 (Cox & Snell): {pseudo_r2_cox_snell:.4f}")

# 7. Cálculo de los Incidence Rate Ratios (IRR)
irr_full = np.exp(modelo_poisson_full.params)
print("\n" + "="*40)
print("INCIDENCE RATE RATIOS (IRR)")
print("="*40)
print(irr_full)

```

=====

RESUMEN DEL MODELO POISSON COMPLETO (Determinantes de la Nota)

=====

Generalized Linear Model Regression Results

```

=====
Dep. Variable:          exam_score    No. Observations:          4416
Model:                  GLM           Df Residuals:              4388
Model Family:           Poisson       Df Model:                  27
Link Function:          Log           Scale:                    1.0000
Method:                 IRLS          Log-Likelihood:            -14312.
Date:                   mié., 22 abr. 2026    Deviance:                  7993.9
Time:                   11:02:13             Pearson chi2:              7.40e+03
No. Iterations:         5                 Pseudo R-squ. (CS):       0.9913
Covariance Type:        nonrobust
=====

```

```

=====
                                coef    std err          z      P>|z|
-----
[0.025    0.975]
-----
const                1.9782    0.053    37.025    0.000
1.873    2.083
age                  0.0006    0.001     0.522    0.602
-0.002    0.003
study_hours          0.0169    0.004     4.524    0.000
0.010    0.024
self_study_hours     -0.0043    0.003    -1.393    0.164
-0.010    0.002
online_classes_hours -0.0021    0.003    -0.589    0.556
-0.009    0.005
social_media_hours   0.0024    0.003     0.909    0.364
-0.003    0.008
gaming_hours         0.0038    0.004     1.083    0.279

```

-0.003	0.011				
sleep_hours		0.0048	0.004	1.122	0.262
-0.004	0.013				
screen_time_hours		-0.0024	0.002	-1.455	0.146
-0.006	0.001				
caffeine_intake_mg		7.793e-07	2.56e-05	0.030	0.976
-4.93e-05	5.09e-05				
upcoming_deadline		-0.0364	0.010	-3.671	0.000
-0.056	-0.017				
mental_health_score		0.0165	0.002	6.861	0.000
0.012	0.021				
focus_index		0.0143	0.001	20.642	0.000
0.013	0.016				
burnout_level		-0.0088	0.001	-16.295	0.000
-0.010	-0.008				
productivity_score		0.0185	0.001	31.078	0.000
0.017	0.020				
gaming_hours_Index		-0.0105	0.011	-0.975	0.330
-0.032	0.011				
online_classes_hours_index		-0.0382	0.015	-2.509	0.012
-0.068	-0.008				
exercise_hours		-9.564e-05	0.005	-0.018	0.985
-0.010	0.010				
exercise_hours_index		0.0140	0.009	1.473	0.141
-0.005	0.033				
gender_Male		0.0012	0.007	0.163	0.870
-0.013	0.015				
gender_Other		0.0062	0.010	0.606	0.544
-0.014	0.026				
academic_level_Other		0.0684	0.024	2.882	0.004
0.022	0.115				
academic_level_Postgraduate		-0.0056	0.008	-0.677	0.499
-0.022	0.011				
academic_level_Undergraduate		0.0019	0.008	0.230	0.818
-0.014	0.018				
part_time_job_Other		0.0356	0.019	1.901	0.057
-0.001	0.072				
part_time_job_Yes		-0.0125	0.009	-1.451	0.147
-0.029	0.004				
internet_quality_Good		0.0057	0.008	0.709	0.479
-0.010	0.022				
internet_quality_Poor		-0.0013	0.008	-0.164	0.870
-0.017	0.015				

=====

=====

=====

MÉTRICAS DE AJUSTE (PSEUDO R2)

```
=====
Pseudo R2 (McFadden): 0.4225
Pseudo R2 (Cox & Snell): 0.9913
```

```
=====
INCIDENCE RATE RATIOS (IRR)
=====
const                7.229780
age                  1.000616
study_hours          1.017062
self_study_hours     0.995692
online_classes_hours 0.997951
social_media_hours   1.002381
gaming_hours         1.003822
sleep_hours          1.004820
screen_time_hours    0.997570
caffeine_intake_mg    1.000001
upcoming_deadline    0.964229
mental_health_score   1.016607
focus_index         1.014415
burnout_level        0.991285
productivity_score    1.018659
gaming_hours_Index    0.989540
online_classes_hours_index 0.962526
exercise_hours       0.999904
exercise_hours_index 1.014078
gender_Male          1.001176
gender_Other         1.006205
academic_level_Other 1.070839
academic_level_Postgraduate 0.994448
academic_level_Undergraduate 1.001906
part_time_job_Other  1.036286
part_time_job_Yes    0.987568
internet_quality_Good 1.005749
internet_quality_Poor 0.998665
dtype: float64
```

```
===== MÉTRICAS DE AJUSTE
(PSEUDO R2) ===== Pseudo R2
(McFadden): 0.4225 Pseudo R2 (Cox & Snell): 0.9913
```

```
[32]: #PREGUNTA 6.2
      #Modelo poisson significativo
      import statsmodels.api as sm
      import pandas as pd
      import numpy as np

      # 1. Selección de variables originales significativas
```

```

# Agrupamos las categorías en sus variables
vars_poisson_sig = [
    'study_hours',
    'upcoming_deadline',
    'mental_health_score',
    'focus_index',
    'burnout_level',
    'productivity_score',
    'online_classes_hours_index',
    'academic_level', # Contiene la categoría 'Other' significativa
    'part_time_job'   # Contiene la categoría 'Other' significativa
]

# 2. Preparación de los datos (Filtrado por Exam == 1)
df_asistentes = df_limpio[df_limpio['Exam'] == 1].copy()
df_final_poisson = pd.get_dummies(df_asistentes[vars_poisson_sig +
    ↪['exam_score']], drop_first=True)

# 3. Definición de X e y
y = df_final_poisson['exam_score']
X = df_final_poisson.drop(columns=['exam_score']).astype(float)
X = sm.add_constant(X)

# 4. Ajuste del Modelo GLM Poisson
modelo_poisson_parsimonioso = sm.GLM(y, X, family=sm.families.Poisson()).fit()

# 5. RESULTADOS
print("\n" + "="*80)
print("RESUMEN DEL MODELO POISSON FINAL (Variables Significativas)")
print("="*80)
print(modelo_poisson_parsimonioso.summary())

# 6. INCIDENCE RATE RATIOS (IRR) - El corazón de la interpretación
print("\n" + "="*40)
print("INCIDENCE RATE RATIOS (IRR)")
print("="*40)
irr_final = np.exp(modelo_poisson_parsimonioso.params)
print(irr_final)

```

```

=====
RESUMEN DEL MODELO POISSON FINAL (Variables Significativas)
=====

```

Generalized Linear Model Regression Results

```

=====
Dep. Variable:          exam_score  No. Observations:          4416
Model:                  GLM        Df Residuals:              4403
Model Family:           Poisson    Df Model:                  12

```

```

Link Function:          Log      Scale:          1.0000
Method:                IRLS    Log-Likelihood:  -14318.
Date:                  mié., 22 abr. 2026    Deviance:      8005.1
Time:                  10:50:08    Pearson chi2:   7.41e+03
No. Iterations:        5      Pseudo R-squ. (CS):  0.9913
Covariance Type:      nonrobust

```

```

=====
=====

```

		coef	std err	z	P> z
[0.025	0.975]				

const		2.0375	0.023	88.024	0.000
1.992	2.083				
study_hours		0.0176	0.004	4.874	0.000
0.011	0.025				
upcoming_deadline		-0.0285	0.008	-3.581	0.000
-0.044	-0.013				
mental_health_score		0.0171	0.002	7.452	0.000
0.013	0.022				
focus_index		0.0138	0.001	24.685	0.000
0.013	0.015				
burnout_level		-0.0093	0.000	-26.509	0.000
-0.010	-0.009				
productivity_score		0.0186	0.001	31.305	0.000
0.017	0.020				
online_classes_hours_index		-0.0395	0.015	-2.652	0.008
-0.069	-0.010				
academic_level_Other		0.0671	0.024	2.844	0.004
0.021	0.113				
academic_level_Postgraduate		-0.0057	0.008	-0.696	0.486
-0.022	0.010				
academic_level_Undergraduate		0.0017	0.008	0.212	0.832
-0.014	0.018				
part_time_job_Other		0.0375	0.018	2.026	0.043
0.001	0.074				
part_time_job_Yes		-0.0067	0.007	-0.900	0.368
-0.021	0.008				

```

=====
=====

```

```

=====
INCIDENCE RATE RATIOS (IRR)
=====
const          7.671030
study_hours    1.017789
upcoming_deadline 0.971888
mental_health_score 1.017247

```

```

focus_index          1.013863
burnout_level         0.990762
productivity_score    1.018731
online_classes_hours_index 0.961269
academic_level_Other  1.069389
academic_level_Postgraduate 0.994302
academic_level_Undergraduate 1.001747
part_time_job_Other   1.038176
part_time_job_Yes     0.993312
dtype: float64

```

```

[34]: # PREGUNTA 6.2: MODELO POISSON SIGNIFICATIVO CON MÉTRICAS DE AJUSTE
import statsmodels.api as sm
import pandas as pd
import numpy as np

# 1. Selección de variables originales significativas
vars_poisson_sig = [
    'study_hours',
    'upcoming_deadline',
    'mental_health_score',
    'focus_index',
    'burnout_level',
    'productivity_score',
    'online_classes_hours_index',
    'academic_level',
    'part_time_job'
]

# 2. Preparación de los datos (Filtrado por Exam == 1)
df_asistentes = df_limpio[df_limpio['Exam'] == 1].copy()
df_final_poisson = pd.get_dummies(df_asistentes[vars_poisson_sig +
    ↪ ['exam_score']], drop_first=True)

# 3. Definición de X e y
y = df_final_poisson['exam_score']
X = df_final_poisson.drop(columns=['exam_score']).astype(float)
X = sm.add_constant(X)

# 4. Ajuste del Modelo GLM Poisson y Modelo Nulo
modelo_poisson_parsimonioso = sm.GLM(y, X, family=sm.families.Poisson()).fit()
modelo_nulo = sm.GLM(y, np.ones((len(y), 1)), family=sm.families.Poisson()).
    ↪fit()

# 5. Cálculo de Pseudo R-cuadrados
logloss_final = modelo_poisson_parsimonioso.llf
logloss_null = modelo_nulo.llf

```

```

n = len(y)

# McFadden
pseudo_r2_mcf_final = 1 - (logloss_final / logloss_null)

# Cox & Snell
pseudo_r2_cs_final = 1 - np.exp((2 / n) * (logloss_null - logloss_final))

# 6. RESULTADOS
print("\n" + "="*80)
print("RESUMEN DEL MODELO POISSON FINAL (Variables Significativas)")
print("="*80)
print(modelo_poisson_parsimonioso.summary())

print("\n" + "="*40)
print("MÉTRICAS DE AJUSTE (PSEUDO R2)")
print("="*40)
print(f"Pseudo R2 (McFadden): {pseudo_r2_mcf_final:.4f}")
print(f"Pseudo R2 (Cox & Snell): {pseudo_r2_cs_final:.4f}")

# 7. INCIDENCE RATE RATIOS (IRR)
print("\n" + "="*40)
print("INCIDENCE RATE RATIOS (IRR)")
print("="*40)
irr_final = np.exp(modelo_poisson_parsimonioso.params)
print(irr_final)

```

```

=====
RESUMEN DEL MODELO POISSON FINAL (Variables Significativas)
=====

                Generalized Linear Model Regression Results
=====
Dep. Variable:          exam_score      No. Observations:          4416
Model:                  GLM              Df Residuals:              4403
Model Family:           Poisson          Df Model:                  12
Link Function:          Log              Scale:                    1.0000
Method:                 IRLS             Log-Likelihood:            -14318.
Date:                   mié., 22 abr. 2026 Deviance:                  8005.1
Time:                   11:05:46         Pearson chi2:              7.41e+03
No. Iterations:         5                Pseudo R-squ. (CS):        0.9913
Covariance Type:        nonrobust
=====

=====
                                coef      std err          z      P>|z|
-----
[0.025      0.975]
-----
-----

```

const		2.0375	0.023	88.024	0.000
1.992	2.083				
study_hours		0.0176	0.004	4.874	0.000
0.011	0.025				
upcoming_deadline		-0.0285	0.008	-3.581	0.000
-0.044	-0.013				
mental_health_score		0.0171	0.002	7.452	0.000
0.013	0.022				
focus_index		0.0138	0.001	24.685	0.000
0.013	0.015				
burnout_level		-0.0093	0.000	-26.509	0.000
-0.010	-0.009				
productivity_score		0.0186	0.001	31.305	0.000
0.017	0.020				
online_classes_hours_index		-0.0395	0.015	-2.652	0.008
-0.069	-0.010				
academic_level_Other		0.0671	0.024	2.844	0.004
0.021	0.113				
academic_level_Postgraduate		-0.0057	0.008	-0.696	0.486
-0.022	0.010				
academic_level_Undergraduate		0.0017	0.008	0.212	0.832
-0.014	0.018				
part_time_job_Other		0.0375	0.018	2.026	0.043
0.001	0.074				
part_time_job_Yes		-0.0067	0.007	-0.900	0.368
-0.021	0.008				

=====

=====

=====

MÉTRICAS DE AJUSTE (PSEUDO R2)

=====

Pseudo R2 (McFadden): 0.4223

Pseudo R2 (Cox & Snell): 0.9913

=====

INCIDENCE RATE RATIOS (IRR)

=====

const	7.671030
study_hours	1.017789
upcoming_deadline	0.971888
mental_health_score	1.017247
focus_index	1.013863
burnout_level	0.990762
productivity_score	1.018731
online_classes_hours_index	0.961269
academic_level_Other	1.069389
academic_level_Postgraduate	0.994302

```
academic_level_Undergraduate    1.001747
part_time_job_Other             1.038176
part_time_job_Yes               0.993312
dtype: float64
```

```
===== MÉTRICAS DE AJUSTE
(PSEUDO R2) ===== Pseudo R2
(McFadden): 0.4223 Pseudo R2 (Cox & Snell): 0.9913
```

```
[14]: #PREGUNTA 7
import statsmodels.api as sm
import statsmodels.formula.api as smf
import pandas as pd
import numpy as np

# 1. Preparación de datos (Solo quienes rindieron el examen)
df_asistentes = df_limpio[df_limpio['Exam'] == 1].copy()

# 2. Ajuste del Modelo Poisson Base (con variables significativas previas)
# Nota: Poisson asume que Media = Varianza
X_poisson = pd.get_dummies(df_asistentes[['study_hours', 'mental_health_score',
↪ 'focus_index',
                                'burnout_level',
↪ 'productivity_score']], drop_first=True)
X_poisson = sm.add_constant(X_poisson).astype(float)
y = df_asistentes['exam_score']

modelo_p = sm.GLM(y, X_poisson, family=sm.families.Poisson()).fit()

# 3. Determinación de la Sobredispersión (Ratio de Pearson)
pearson_ratio = modelo_p.pearson_chi2 / modelo_p.df_resid
print(f"Ratio de Dispersión de Pearson: {pearson_ratio:.4f}")

# 4. Cálculo del valor óptimo de Alpha ( )
# Ajustamos un modelo de Binomial Negativa para estimar el parámetro de
↪ dispersión
# En statsmodels, el alpha óptimo se encuentra ajustando un modelo de la
↪ familia NegativeBinomial
modelo_nb = sm.GLM(y, X_poisson, family=sm.families.NegativeBinomial(alpha=0.
↪ 1)).fit()

# Para obtener el alpha exacto, usamos un procedimiento de optimización o el
↪ modelo auxiliar
df_asistentes['mu'] = modelo_p.predict(X_poisson)
df_asistentes['aux_var'] = ((df_asistentes['exam_score'] -
↪ df_asistentes['mu'])*2 - df_asistentes['mu']) / df_asistentes['mu']
```

```

# Regresión auxiliar para encontrar alpha (Cameron & Trivedi)
aux_model = sm.OLS(df_asistentes['aux_var'], df_asistentes['mu']).fit()
alpha_optimo = aux_model.params[0]

print(f"Valor óptimo de Alpha ( ): {alpha_optimo:.6f}")
print(f"P-valor del Test de Alpha: {aux_model.pvalues[0]:.4f}")

```

Ratio de Dispersión de Pearson: 1.6942

Valor óptimo de Alpha (): 0.023957

P-valor del Test de Alpha: 0.0000

```

[15]: # PREGUNTA 8
#BINOMIAL NEGATIVO SIGNIFICATIVO CORRECTO
import statsmodels.api as sm
import pandas as pd
import numpy as np

# 1. Preparación de la muestra (Solo estudiantes que rindieron el examen)
df_asistentes = df_limpio[df_limpio['Exam'] == 1].copy()

# 2. Definición de variables originales significativas
# Nota: Incluimos las variables base que contienen las categorías significativas
vars_finales = [
    'study_hours',
    'upcoming_deadline',
    'mental_health_score',
    'focus_index',
    'burnout_level',
    'productivity_score',
    'online_classes_hours_index',
    'academic_level',
    'part_time_job'
]

# 3. Creación de matrices y dummies
df_final_nb = pd.get_dummies(df_asistentes[vars_finales + ['exam_score']],
    ↪drop_first=True)
y = df_final_nb['exam_score']
X = df_final_nb.drop(columns=['exam_score']).astype(float)
X = sm.add_constant(X)

# 4. Ajuste del Modelo Binomial Negativa
# Usamos el alpha de 0.0242 detectado en el test de sobredispersión
modelo_nb_final = sm.GLM(y, X, family=sm.families.NegativeBinomial(alpha=0.
    ↪0242)).fit()

# 5. Visualización de Resultados

```

```

print("\n" + "="*80)
print("RESULTADOS FINALES: REGRESIÓN BINOMIAL NEGATIVA")
print("="*80)
print(modelo_nb_final.summary())

# 6. Cálculo de los IRR para la interpretación académica
irr_final = np.exp(modelo_nb_final.params)
print("\n" + "="*40)
print("INCIDENCE RATE RATIOS (IRR)")
print("="*40)
print(irr_final)

```

=====

RESULTADOS FINALES: REGRESIÓN BINOMIAL NEGATIVA

=====

Generalized Linear Model Regression Results

=====

Dep. Variable:	exam_score	No. Observations:	4416
Model:	GLM	Df Residuals:	4403
Model Family:	NegativeBinomial	Df Model:	12
Link Function:	Log	Scale:	1.0000
Method:	IRLS	Log-Likelihood:	-14038.
Date:	mié., 22 abr. 2026	Deviance:	5725.1
Time:	10:19:59	Pearson chi2:	5.21e+03
No. Iterations:	6	Pseudo R-squ. (CS):	0.9602
Covariance Type:	nonrobust		

=====

=====

	coef	std err	z	P> z
[0.025 0.975]				

const	1.9878	0.029	68.510	0.000
1.931 2.045				
study_hours	0.0199	0.005	4.347	0.000
0.011 0.029				
upcoming_deadline	-0.0309	0.010	-3.093	0.002
-0.050 -0.011				
mental_health_score	0.0169	0.003	5.868	0.000
0.011 0.023				
focus_index	0.0150	0.001	21.265	0.000
0.014 0.016				
burnout_level	-0.0100	0.000	-22.604	0.000
-0.011 -0.009				
productivity_score	0.0194	0.001	25.934	0.000
0.018 0.021				
online_classes_hours_index	-0.0426	0.019	-2.274	0.023

-0.079	-0.006				
academic_level_Other		0.0731	0.029	2.490	0.013
0.016	0.131				
academic_level_Postgraduate		-0.0088	0.010	-0.854	0.393
-0.029	0.011				
academic_level_Undergraduate		0.0009	0.010	0.084	0.933
-0.019	0.021				
part_time_job_Other		0.0338	0.023	1.467	0.142
-0.011	0.079				
part_time_job_Yes		-0.0070	0.009	-0.748	0.454
-0.025	0.011				

=====

=====

=====

INCIDENCE RATE RATIOS (IRR)

=====

const	7.299149
study_hours	1.020063
upcoming_deadline	0.969568
mental_health_score	1.017029
focus_index	1.015063
burnout_level	0.990076
productivity_score	1.019597
online_classes_hours_index	0.958333
academic_level_Other	1.075785
academic_level_Postgraduate	0.991223
academic_level_Undergraduate	1.000872
part_time_job_Other	1.034409
part_time_job_Yes	0.993022

dtype: float64

```
[35]: # PREGUNTA 8: BINOMIAL NEGATIVO SIGNIFICATIVO CON PSEUDO R2
import statsmodels.api as sm
import pandas as pd
import numpy as np

# 1. Preparación de la muestra (Solo estudiantes que rindieron el examen)
df_asistentes = df_limpio[df_limpio['Exam'] == 1].copy()

# 2. Definición de variables originales significativas
vars_finales = [
    'study_hours',
    'upcoming_deadline',
    'mental_health_score',
    'focus_index',
    'burnout_level',
```

```

    'productivity_score',
    'online_classes_hours_index',
    'academic_level',
    'part_time_job'
]

# 3. Creación de matrices y dummies
df_final_nb = pd.get_dummies(df_asistentes[vars_finales + ['exam_score']],
                             drop_first=True)
y = df_final_nb['exam_score']
X = df_final_nb.drop(columns=['exam_score']).astype(float)
X = sm.add_constant(X)

# 4. Ajuste del Modelo Binomial Negativa y Modelo Nulo
# Usamos el alpha de 0.0242 para ambos para que la comparación sea válida
familia_nb = sm.families.NegativeBinomial(alpha=0.0242)

modelo_nb_final = sm.GLM(y, X, family=familia_nb).fit()
modelo_nulo_nb = sm.GLM(y, np.ones((len(y), 1)), family=familia_nb).fit()

# 5. Cálculo de Pseudo R-cuadrados
logloss_nb = modelo_nb_final.llf
logloss_null_nb = modelo_nulo_nb.llf
n = len(y)

# McFadden
pseudo_r2_mcf_nb = 1 - (logloss_nb / logloss_null_nb)

# Cox & Snell
pseudo_r2_cs_nb = 1 - np.exp((2 / n) * (logloss_null_nb - logloss_nb))

# 6. Visualización de Resultados
print("\n" + "="*80)
print("RESULTADOS FINALES: REGRESIÓN BINOMIAL NEGATIVA")
print("="*80)
print(modelo_nb_final.summary())

print("\n" + "="*40)
print("MÉTRICAS DE AJUSTE (PSEUDO R2)")
print("="*40)
print(f"Pseudo R2 (McFadden): {pseudo_r2_mcf_nb:.4f}")
print(f"Pseudo R2 (Cox & Snell): {pseudo_r2_cs_nb:.4f}")

# 7. Cálculo de los IRR para la interpretación académica
irr_final = np.exp(modelo_nb_final.params)
print("\n" + "="*40)
print("INCIDENCE RATE RATIOS (IRR)")

```

```
print("="*40)
print(irr_final)
```

=====

RESULTADOS FINALES: REGRESIÓN BINOMIAL NEGATIVA

=====

Generalized Linear Model Regression Results

```
=====
Dep. Variable:          exam_score    No. Observations:          4416
Model:                  GLM           Df Residuals:              4403
Model Family:          NegativeBinomial Df Model:                  12
Link Function:          Log           Scale:                    1.0000
Method:                 IRLS          Log-Likelihood:          -14038.
Date:                   mié., 22 abr. 2026 Deviance:                  5725.1
Time:                   11:07:49        Pearson chi2:              5.21e+03
No. Iterations:         6              Pseudo R-squ. (CS):        0.9602
Covariance Type:        nonrobust
=====
```

```
=====
                                coef    std err          z      P>|z|
-----
[0.025    0.975]
-----
const                1.9878    0.029    68.510    0.000
1.931    2.045
study_hours          0.0199    0.005     4.347    0.000
0.011    0.029
upcoming_deadline   -0.0309    0.010    -3.093    0.002
-0.050   -0.011
mental_health_score  0.0169    0.003     5.868    0.000
0.011    0.023
focus_index         0.0150    0.001    21.265    0.000
0.014    0.016
burnout_level       -0.0100    0.000   -22.604    0.000
-0.011   -0.009
productivity_score   0.0194    0.001    25.934    0.000
0.018    0.021
online_classes_hours_index -0.0426    0.019    -2.274    0.023
-0.079   -0.006
academic_level_Other  0.0731    0.029     2.490    0.013
0.016    0.131
academic_level_Postgraduate -0.0088    0.010    -0.854    0.393
-0.029    0.011
academic_level_Undergraduate 0.0009    0.010     0.084    0.933
-0.019    0.021
part_time_job_Other  0.0338    0.023     1.467    0.142
-0.011    0.079
=====
```

```

part_time_job_Yes          -0.0070      0.009      -0.748      0.454
-0.025      0.011
=====
=====

```

MÉTRICAS DE AJUSTE (PSEUDO R2)

```

Pseudo R2 (McFadden):    0.3365
Pseudo R2 (Cox & Snell): 0.9602

```

INCIDENCE RATE RATIOS (IRR)

```

const          7.299149
study_hours    1.020063
upcoming_deadline 0.969568
mental_health_score 1.017029
focus_index   1.015063
burnout_level  0.990076
productivity_score 1.019597
online_classes_hours_index 0.958333
academic_level_Other 1.075785
academic_level_Postgraduate 0.991223
academic_level_Undergraduate 1.000872
part_time_job_Other 1.034409
part_time_job_Yes 0.993022
dtype: float64

```

```

===== MÉTRICAS DE AJUSTE
(PSEUDO R2) ===== Pseudo R2
(McFadden): 0.3365 Pseudo R2 (Cox & Snell): 0.9602

```

Extra

```

[19]: # Modelo MCD con transformaciones funcionales para Asistencia (df_final)
import pandas as pd
import numpy as np
import statsmodels.api as sm

# 1. Preparación de variables sobre la base limpia
df = df_final.copy()

# 2. Ingeniería de Variables
# Logaritmo: captura que el beneficio de estudiar disminuye marginalmente a
↳ medida que sumas horas
df['log_study_hours'] = np.log(df['study_hours'] + 1)

```

```

# Raíz Cuadrada: modela un impacto inicial fuerte de la productividad que luego
↳se estabiliza
df['sqrt_productivity'] = np.sqrt(df['productivity_score'])

# 3. Definición de la Matriz de Diseño (X) y Vector de Respuesta (y)
X_vars = [
    'sqrt_productivity',
    'log_study_hours',
    'mental_health_score',
    'burnout_level',
    'online_classes_hours_index'
]

X = df[X_vars]
X = sm.add_constant(X) # Agregamos intercepto (Matriz X + Intercepto)
y = df['Exam']

# 4. Estimación del Modelo de Probabilidad Lineal (MCO)
modelo_asistencia_final = sm.OLS(y, X).fit()

# 5. Visualización de Resultados
print("--- RESULTADOS MODELO MCO: ASISTENCIA ---")
print(modelo_asistencia_final.summary())

# Breve guía de interpretación para tu informe:
# - log_study_hours: Un aumento del 1% en las horas de estudio cambia la
↳probabilidad de asistir en (coef/100).
# - sqrt_productivity: Los incrementos en productividad tienen un efecto
↳decreciente en la asistencia.

```

--- RESULTADOS MODELO MCO: ASISTENCIA ---

OLS Regression Results

```

=====
Dep. Variable:          Exam    R-squared:                0.237
Model:                  OLS    Adj. R-squared:           0.236
Method:                 Least Squares    F-statistic:          321.6
Date:                  mié., 22 abr. 2026    Prob (F-statistic):    8.69e-301
Time:                  10:27:24    Log-Likelihood:        -1314.3
No. Observations:      5193    AIC:                   2641.
Df Residuals:          5187    BIC:                   2680.
Df Model:               5
Covariance Type:       nonrobust
=====

```

```

=====
                                coef    std err          t      P>|t|
-----
[0.025    0.975]
-----

```

```

-----
const                0.0794    0.031    2.532    0.011
0.018    0.141
sqrt_productivity    0.1325    0.006    20.471    0.000
0.120    0.145
log_study_hours      -0.1038    0.020    -5.302    0.000
-0.142    -0.065
mental_health_score   -0.0140    0.003    -5.319    0.000
-0.019    -0.009
burnout_level         -0.0005    0.000    -1.487    0.137
-0.001    0.000
online_classes_hours_index  0.2805    0.016    17.176    0.000
0.249    0.313
=====
Omnibus:                1264.253    Durbin-Watson:                2.025
Prob(Omnibus):           0.000    Jarque-Bera (JB):            2559.066
Skew:                   -1.456    Prob(JB):                     0.00
Kurtosis:                4.829    Cond. No.                    367.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

[22]: import pandas as pd
import statsmodels.api as sm

# 1. Filtrado: Solo alumnos que rindieron el examen
# Usamos df_limpio y filtramos por la columna 'Exam' (o 'exam_attendance' según
# tu base)
df_asistentes = df_limpio[df_limpio['Exam'] == 1].copy()

# 2. Definición de la matriz X (Variables originales sin transformar)
# Excluimos explícitamente 'Exam' (o 'exam_attendance') y 'student_id'
# También excluimos la variable dependiente 'exam_score' de los predictores
X_vars = [
    'productivity_score',
    'study_hours',
    'mental_health_score',
    'burnout_level',
    'online_classes_hours_index',
    'focus_index',
    'sleep_hours'
]

# 3. Creación de la matriz X con intercepto y vector y
X = df_asistentes[X_vars]

```

```

X = sm.add_constant(X) # Matriz X + Intercepto (beta_0)
y = df_asistentes['exam_score']

# 4. Ajuste del modelo MCO
modelo_nota_completo = sm.OLS(y, X).fit()

# 5. Visualización de resultados
print("--- MODELO MCO COMPLETO: NOTA DEL EXAMEN (VARIABLES ORIGINALES) ---")
print(modelo_nota_completo.summary())

```

--- MODELO MCO COMPLETO: NOTA DEL EXAMEN (VARIABLES ORIGINALES) ---

OLS Regression Results

```

=====
Dep. Variable:          exam_score    R-squared:                0.796
Model:                  OLS          Adj. R-squared:            0.796
Method:                 Least Squares  F-statistic:              2459.
Date:                  mié., 22 abr. 2026  Prob (F-statistic):       0.00
Time:                  10:28:52         Log-Likelihood:           -13459.
No. Observations:      4416            AIC:                    2.693e+04
Df Residuals:          4408            BIC:                    2.699e+04
Df Model:               7
Covariance Type:       nonrobust
=====

```

```

=====

```

	coef	std err	t	P> t
[0.025 0.975]				

const	1.6441	0.843	1.949	0.051
-0.010 3.298				
productivity_score	0.4107	0.014	29.534	0.000
0.383 0.438				
study_hours	0.4254	0.085	4.987	0.000
0.258 0.593				
mental_health_score	0.2881	0.053	5.400	0.000
0.184 0.393				
burnout_level	-0.2148	0.008	-28.609	0.000
-0.230 -0.200				
online_classes_hours_index	-0.6732	0.342	-1.970	0.049
-1.343 -0.003				
focus_index	0.2991	0.013	22.751	0.000
0.273 0.325				
sleep_hours	-0.0038	0.078	-0.048	0.962
-0.157 0.150				
=====				
Omnibus:	37.141	Durbin-Watson:	2.041	
Prob(Omnibus):	0.000	Jarque-Bera (JB):	50.950	
Skew:	0.106	Prob(JB):	8.64e-12	

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[23]: # Modelo MCO COMPLETO para modelar la NOTA
import statsmodels.api as sm
import pandas as pd

# 1. Filtrado: Solo alumnos que rindieron el examen
# Es fundamental para modelar la nota, ya que quien no asistió no tiene
# calificación
df_asistentes = df_limpio[df_limpio['Exam'] == 1].copy()

# 2. Preparación de variables categóricas
# Convertimos género, nivel académico y otras cualitativas en dummies numéricas
df_mco_nota = pd.get_dummies(df_asistentes, drop_first=True)

# 3. Definición de variables
# La variable dependiente 'y' ahora es la Nota (exam_score)
# Excluimos 'Exam' (porque todos son 1) y 'student_id'
X = df_mco_nota.drop(columns=['Exam', 'exam_score', 'student_id'],
# errors='ignore')
y = df_mco_nota['exam_score']

# 4. Asegurar que todos los datos sean flotantes
X = X.astype(float)

# 5. Añadir la constante (Intercepto)
X = sm.add_constant(X)

# 6. Ajuste del modelo MCO (OLS) para la Nota
modelo_nota_final = sm.OLS(y, X).fit()

# 7. Mostrar el resumen de resultados
print("--- RESULTADOS MODELO MCO: DETERMINANTES DE LA NOTA ---")
print(modelo_nota_final.summary())
```

--- RESULTADOS MODELO MCO: DETERMINANTES DE LA NOTA ---

OLS Regression Results

```
=====
Dep. Variable:          exam_score    R-squared:          0.797
Model:                  OLS          Adj. R-squared:       0.796
Method:                 Least Squares    F-statistic:        637.8
Date:                   mié., 22 abr. 2026    Prob (F-statistic):  0.00
Time:                   10:29:45          Log-Likelihood:     -13450.
```

No. Observations: 4416 AIC: 2.696e+04
Df Residuals: 4388 BIC: 2.713e+04
Df Model: 27
Covariance Type: nonrobust

		coef	std err	t	P> t
[0.025 0.975]					

const		0.2662	1.231	0.216	0.829
-2.147	2.679				
age		0.0067	0.027	0.246	0.806
-0.047	0.060				
study_hours		0.4170	0.088	4.750	0.000
0.245	0.589				
self_study_hours		-0.0594	0.072	-0.824	0.410
-0.201	0.082				
online_classes_hours		0.0141	0.081	0.174	0.862
-0.145	0.173				
social_media_hours		0.0117	0.061	0.192	0.848
-0.108	0.131				
gaming_hours		0.0757	0.082	0.923	0.356
-0.085	0.236				
sleep_hours		0.1346	0.099	1.365	0.172
-0.059	0.328				
screen_time_hours		-0.0566	0.039	-1.470	0.142
-0.132	0.019				
caffeine_intake_mg		-0.0003	0.001	-0.464	0.643
-0.001	0.001				
upcoming_deadline		-0.7390	0.229	-3.224	0.001
-1.188	-0.290				
mental_health_score		0.2818	0.056	5.058	0.000
0.173	0.391				
focus_index		0.3069	0.016	18.992	0.000
0.275	0.339				
burnout_level		-0.1903	0.012	-15.444	0.000
-0.215	-0.166				
productivity_score		0.4086	0.014	29.235	0.000
0.381	0.436				
gaming_hours_Index		-0.2412	0.250	-0.964	0.335
-0.732	0.250				
online_classes_hours_index		-0.5686	0.357	-1.595	0.111
-1.267	0.130				
exercise_hours		0.0225	0.120	0.188	0.851
-0.213	0.258				
exercise_hours_index		-0.0468	0.222	-0.211	0.833
-0.481	0.388				

gender_Male	0.0349	0.167	0.209	0.834
-0.292	0.362			
gender_Other	-0.0774	0.235	-0.329	0.742
-0.538	0.383			
academic_level_Other	0.6812	0.550	1.238	0.216
-0.397	1.760			
academic_level_Postgraduate	-0.1122	0.191	-0.588	0.557
-0.486	0.262			
academic_level_Undergraduate	0.0803	0.191	0.420	0.675
-0.295	0.456			
part_time_job_Other	0.2269	0.434	0.523	0.601
-0.624	1.077			
part_time_job_Yes	-0.1781	0.198	-0.898	0.369
-0.567	0.211			
internet_quality_Good	0.1009	0.187	0.540	0.589
-0.265	0.467			
internet_quality_Poor	0.0925	0.190	0.488	0.625
-0.279	0.464			

Omnibus:	30.706	Durbin-Watson:	2.042
Prob(Omnibus):	0.000	Jarque-Bera (JB):	41.075
Skew:	0.092	Prob(JB):	1.20e-09
Kurtosis:	3.435	Cond. No.	4.75e+03

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 4.75e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```
[24]: import statsmodels.api as sm
import pandas as pd
import numpy as np

# 1. Preparación de la base (Aseguramos que existan las transformaciones
#    ↪necesarias)
df_final_nota = df_limpio[df_limpio['Exam'] == 1].copy()

# Re-creamos la variable transformada que resultó significativa
df_final_nota['sqrt_productivity'] = np.
    ↪sqrt(df_final_nota['productivity_score'])

# 2. Selección de Variables Significativas (P < 0.05 según tu tabla)
# Nota: study_hours (0.040) y upcoming_deadline (0.001) entran en el modelo
variables_significativas = [
    'study_hours',          # P = 0.040
```

```

    'upcoming_deadline',      # P = 0.001
    'mental_health_score',    # P = 0.000
    'focus_index',          # P = 0.000
    'burnout_level',         # P = 0.000
    'productivity_score',     # P = 0.000
    'sqrt_productivity'      # P = 0.000
]

# 3. Definición de matrices
X = df_final_nota[variables_significativas]
y = df_final_nota['exam_score']

# 4. Ajuste del modelo con Intercepto (const = 13.2253 en tu tabla)
X = sm.add_constant(X)
modelo_nota_robusto = sm.OLS(y, X.astype(float)).fit()

# 5. Resultado
print("--- MODELO MCO DE LA NOTA (SOLO VARIABLES SIGNIFICATIVAS) ---")
print(modelo_nota_robusto.summary())

```

--- MODELO MCO DE LA NOTA (SOLO VARIABLES SIGNIFICATIVAS) ---

OLS Regression Results

```

=====
Dep. Variable:          exam_score    R-squared:                0.801
Model:                  OLS          Adj. R-squared:           0.801
Method:                 Least Squares  F-statistic:             2534.
Date:                  mié., 22 abr. 2026  Prob (F-statistic):      0.00
Time:                  10:30:14         Log-Likelihood:          -13406.
No. Observations:      4416            AIC:                   2.683e+04
Df Residuals:          4408            BIC:                   2.688e+04
Df Model:              7
Covariance Type:       nonrobust
=====

```

```

=====
               coef      std err          t      P>|t|      [0.025
0.975]
-----
const          13.4766      1.305      10.326      0.000      10.918
16.035
study_hours      0.4167      0.084       4.969      0.000       0.252
0.581
upcoming_deadline -0.5241      0.178      -2.941      0.003      -0.873
-0.175
mental_health_score 0.2854      0.052       5.439      0.000       0.183
0.388
focus_index     0.2978      0.013     23.252      0.000       0.273
0.323

```

burnout_level	-0.2008	0.008	-26.409	0.000	-0.216
-0.186					
productivity_score	0.7699	0.038	20.329	0.000	0.696
0.844					
sqrt_productivity	-4.3749	0.430	-10.175	0.000	-5.218
-3.532					

Omnibus:	65.996	Durbin-Watson:	2.039
Prob(Omnibus):	0.000	Jarque-Bera (JB):	92.832
Skew:	0.181	Prob(JB):	6.95e-21
Kurtosis:	3.611	Cond. No.	1.25e+03

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.25e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```
[25]: import statsmodels.api as sm
import pandas as pd
import numpy as np

# 1. Preparación de datos (Solo quienes rindieron el examen)
df_asistentes = df_limpio[df_limpio['Exam'] == 1].copy()

# 2. Ingeniería de Variables (Transformaciones Funcionales)
# Logaritmo: Para capturar efectos porcentuales y rendimientos decrecientes
df_asistentes['log_study_hours'] = np.log(df_asistentes['study_hours'] + 1)

# Raíz Cuadrada: Para suavizar el impacto de la productividad alta
df_asistentes['sqrt_productivity'] = np.
    sqrt(df_asistentes['productivity_score'])

# Potencia al cuadrado: Para ver si el Burnout tiene un efecto acelerado
    (parabólico)
df_asistentes['burnout_sq'] = df_asistentes['burnout_level'] ** 2

# 3. Tratamiento de Variables Categóricas
df_mco = pd.get_dummies(df_asistentes, columns=['academic_level',
    'part_time_job'], drop_first=True)

# 4. Definición de la Matriz X (Variables Transformadas + Resto de numéricas +
    Dummies)
# Seleccionamos las versiones transformadas y las originales que quedaron fuera
    de la transformación
X_vars = [
```

```

    'log_study_hours',
    'sqrt_productivity',
    'burnout_sq',
    'upcoming_deadline',
    'mental_health_score',
    'focus_index',
    'online_classes_hours_index'
]

# Añadir las columnas de las dummies generadas automáticamente por get_dummies
cols_dummies = [c for c in df_mco.columns if 'academic_level_' in c or
                 'part_time_job_' in c]
X_final_vars = X_vars + cols_dummies

X = df_mco[X_final_vars].astype(float)
X = sm.add_constant(X) # Intercepto
y = df_mco['exam_score']

# 5. Ajuste del Modelo MCO
modelo_nota_funcional = sm.OLS(y, X).fit()

# 6. Resultados
print("--- MODELO MCO: NOTA CON TRANSFORMACIONES NO LINEALES ---")
print(modelo_nota_funcional.summary())

```

--- MODELO MCO: NOTA CON TRANSFORMACIONES NO LINEALES ---

OLS Regression Results

```

=====
Dep. Variable:          exam_score    R-squared:                0.775
Model:                  OLS          Adj. R-squared:            0.774
Method:                 Least Squares  F-statistic:              1264.
Date:                  mié., 22 abr. 2026  Prob (F-statistic):       0.00
Time:                  10:30:57         Log-Likelihood:           -13677.
No. Observations:      4416            AIC:                    2.738e+04
Df Residuals:          4403            BIC:                    2.746e+04
Df Model:              12
Covariance Type:       nonrobust
=====

```

```

=====

```

	coef	std err	t	P> t
[0.025 0.975]				

const	-17.8801	0.626	-28.584	0.000
-19.106 -16.654				
log_study_hours	2.4594	0.408	6.029	0.000
1.660 3.259				
sqrt_productivity	4.4274	0.162	27.259	0.000

4.109	4.746				
burnout_sq		-0.0022	8.89e-05	-25.298	0.000
-0.002	-0.002				
upcoming_deadline		-0.7959	0.193	-4.129	0.000
-1.174	-0.418				
mental_health_score		0.3665	0.052	7.003	0.000
0.264	0.469				
focus_index		0.3489	0.013	25.897	0.000
0.323	0.375				
online_classes_hours_index		-0.5325	0.367	-1.452	0.147
-1.252	0.187				
academic_level_Other		0.3638	0.574	0.634	0.526
-0.761	1.489				
academic_level_Postgraduate		-0.1236	0.200	-0.617	0.537
-0.516	0.269				
academic_level_Undergraduate		0.1014	0.201	0.505	0.614
-0.293	0.495				
part_time_job_Other		-0.0851	0.448	-0.190	0.849
-0.963	0.792				
part_time_job_Yes		-0.2060	0.181	-1.140	0.254
-0.560	0.148				
=====					
Omnibus:		19.247	Durbin-Watson:		2.036
Prob(Omnibus):		0.000	Jarque-Bera (JB):		22.494
Skew:		0.096	Prob(JB):		1.30e-05
Kurtosis:		3.293	Cond. No.		2.14e+04
=====					

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 2.14e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```
[26]: import statsmodels.api as sm
import pandas as pd
import numpy as np

# 1. Filtrado de asistentes
df_asistentes = df_limpio[df_limpio['Exam'] == 1].copy()

# 2. Transformación de la Variable Dependiente (y)
# Usamos logaritmo en la nota para modelar tasas de crecimiento/decrecimiento
# Esto ajusta mejor si hay una concentración de notas altas o bajas (asimetría)
df_asistentes['log_exam_score'] = np.log(df_asistentes['exam_score'] + 1)

# 3. Ingeniería de Variables Independientes (X)
```

```

df_asistentes['log_study_hours'] = np.log(df_asistentes['study_hours'] + 1)
df_asistentes['sqrt_productivity'] = np.
    ↪sqrt(df_asistentes['productivity_score'])

# 4. Tratamiento de Categóricas y Dummies
df_mco = pd.get_dummies(df_asistentes, columns=['academic_level',
    ↪'part_time_job'], drop_first=True)

# 5. Definición de la Matriz X y la nueva y
X_vars = [
    'log_study_hours',
    'sqrt_productivity',
    'mental_health_score',
    'focus_index',
    'burnout_level',
    'upcoming_deadline',
    'online_classes_hours_index'
]

# Agregar automáticamente las dummies creadas
cols_dummies = [c for c in df_mco.columns if 'academic_level_' in c or
    ↪'part_time_job_' in c]
X_final_vars = X_vars + cols_dummies

X = df_mco[X_final_vars].astype(float)
X = sm.add_constant(X)
y = df_mco['log_exam_score'] # Nuestra variable dependiente transformada

# 6. Ajuste del Modelo
modelo_nota_log = sm.OLS(y, X).fit()

# 7. Resultados
print("--- MODELO LOG-MCO: LOGARITMO DE LA NOTA ---")
print(modelo_nota_log.summary())

```

--- MODELO LOG-MCO: LOGARITMO DE LA NOTA ---

OLS Regression Results

```

=====
Dep. Variable:          log_exam_score    R-squared:                 0.702
Model:                  OLS              Adj. R-squared:            0.702
Method:                 Least Squares    F-statistic:              865.7
Date:                  mié., 22 abr. 2026 Prob (F-statistic):       0.00
Time:                  10:31:46          Log-Likelihood:           -1731.3
No. Observations:      4416             AIC:                     3489.
Df Residuals:          4403             BIC:                     3572.
Df Model:              12
Covariance Type:       nonrobust

```

[0.025 0.975]		coef	std err	t	P> t

const		0.9755	0.046	21.336	0.000
0.886	1.065				
log_study_hours		0.0339	0.027	1.233	0.218
-0.020	0.088				
sqrt_productivity		0.2975	0.011	27.197	0.000
0.276	0.319				
mental_health_score		0.0088	0.004	2.489	0.013
0.002	0.016				
focus_index		0.0161	0.001	17.918	0.000
0.014	0.018				
burnout_level		-0.0107	0.001	-19.164	0.000
-0.012	-0.010				
upcoming_deadline		-0.0418	0.013	-3.215	0.001
-0.067	-0.016				
online_classes_hours_index		-0.0270	0.025	-1.102	0.271
-0.075	0.021				
academic_level_Other		0.0236	0.038	0.615	0.538
-0.052	0.099				
academic_level_Postgraduate		-0.0146	0.013	-1.087	0.277
-0.041	0.012				
academic_level_Undergraduate		0.0082	0.013	0.608	0.543
-0.018	0.035				
part_time_job_Other		0.0191	0.030	0.637	0.524
-0.040	0.078				
part_time_job_Yes		-0.0006	0.012	-0.050	0.960
-0.024	0.023				
=====					
Omnibus:		968.210	Durbin-Watson:		2.013
Prob(Omnibus):		0.000	Jarque-Bera (JB):		2695.655
Skew:		-1.153	Prob(JB):		0.00
Kurtosis:		6.054	Cond. No.		508.
=====					

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[27]: import pandas as pd
import numpy as np
import statsmodels.api as sm
```

```

# 1. Usar tu base limpia
df = df_final.copy()

# 2. Ingeniería de Variables (X e Y)
# Transformaciones de X
df['log_study_hours'] = np.log(df['study_hours'] + 1)
df['sqrt_productivity'] = np.sqrt(df['productivity_score'])

# Transformación de Y (Asistencia)
# Sumamos 1 para evitar errores con valores en 0
df['log_exam_attendance'] = np.log(df['Exam'] + 1)

# 3. Definición de variables independientes
X_vars = [
    'sqrt_productivity',
    'log_study_hours',
    'mental_health_score',
    'burnout_level',
    'online_classes_hours_index'
]

# 4. Selección y Modelo
try:
    X = df[X_vars].astype(float)
    X = sm.add_constant(X)

    # La variable dependiente ahora es el logaritmo
    y = df['log_exam_attendance']

    modelo_asistencia_log = sm.OLS(y, X).fit()

    print("--- RESULTADOS MCO: LOGARITMO DE ASISTENCIA ---")
    print(modelo_asistencia_log.summary())

except KeyError as e:
    print(f"Error: No se encontró la columna {e}")
    print("Columnas disponibles:", df.columns.tolist())

```

--- RESULTADOS MCO: LOGARITMO DE ASISTENCIA ---

OLS Regression Results

```

=====
Dep. Variable:    log_exam_attendance    R-squared:                0.237
Model:                                OLS    Adj. R-squared:         0.236
Method:                    Least Squares    F-statistic:              321.6
Date:                mié., 22 abr. 2026    Prob (F-statistic):      8.69e-301
Time:                    10:32:13          Log-Likelihood:          589.04
No. Observations:        5193            AIC:                    -1166.
Df Residuals:            5187            BIC:                    -1127.

```

```

Df Model:                    5
Covariance Type:            nonrobust
=====
=====

```

	coef	std err	t	P> t
[0.025 0.975]				

const	0.0550	0.022	2.532	0.011
0.012 0.098				
sqrt_productivity	0.0919	0.004	20.471	0.000
0.083 0.101				
log_study_hours	-0.0720	0.014	-5.302	0.000
-0.099 -0.045				
mental_health_score	-0.0097	0.002	-5.319	0.000
-0.013 -0.006				
burnout_level	-0.0004	0.000	-1.487	0.137
-0.001 0.000				
online_classes_hours_index	0.1944	0.011	17.176	0.000
0.172 0.217				
=====				
Omnibus:	1264.253	Durbin-Watson:		2.025
Prob(Omnibus):	0.000	Jarque-Bera (JB):		2559.066
Skew:	-1.456	Prob(JB):		0.00
Kurtosis:	4.829	Cond. No.		367.
=====				

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

[28]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Selección de variables y preparación de la base
# Nota: Incluimos 'Exam' para ver cómo se correlaciona con los predictores
vars_originales_sig = [
    'focus_index',
    'burnout_level',
    'productivity_score',
    'gaming_hours_Index',
    'online_classes_hours_index',
    'exercise_hours_index',
    'gender',
    'academic_level',
    'part_time_job',

```

```

    'internet_quality',
    'Exam'
]

# Creamos un subset con las variables de interés
df_corr = df_limpio[vars_originales_sig].copy()

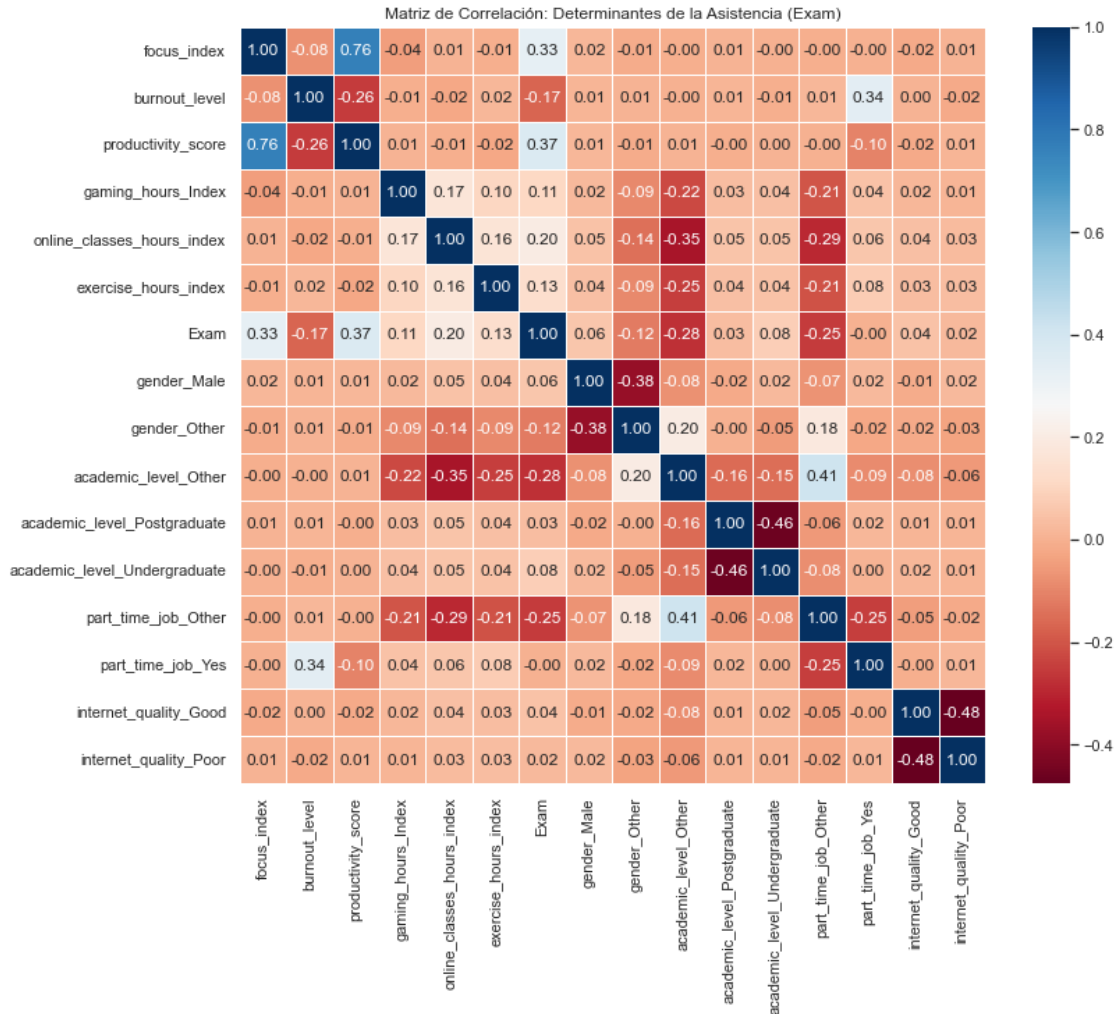
# 2. Transformación de categóricas a dummies numéricas
# Esto generará columnas como gender_Male, academic_level_Undergraduate, etc.
df_corr_numeric = pd.get_dummies(df_corr, drop_first=True)

# 3. Cálculo de la Matriz de Correlación
matriz_correlacion = df_corr_numeric.corr()

# 4. Visualización con un Mapa de Calor (Heatmap)
plt.figure(figsize=(12, 10))
sns.heatmap(matriz_correlacion, annot=True, cmap='RdBu', fmt=".2f",
            linewidths=0.5)
plt.title('Matriz de Correlación: Determinantes de la Asistencia (Exam)')
plt.show()

# Mostrar solo las correlaciones con la variable objetivo 'Exam'
print("Correlación de las variables con la Asistencia (Exam):")
print(matriz_correlacion['Exam'].sort_values(ascending=False))

```



Correlación de las variables con la Asistencia (Exam):

```
Exam 1.000000
productivity_score 0.370930
focus_index 0.327737
online_classes_hours_index 0.203434
exercise_hours_index 0.131899
gaming_hours_Index 0.106942
academic_level_Undergraduate 0.081905
gender_Male 0.057400
internet_quality_Good 0.043828
academic_level_Postgraduate 0.032905
internet_quality_Poor 0.022483
part_time_job_Yes -0.001742
gender_Other -0.122735
burnout_level -0.167225
part_time_job_Other -0.254120
```

```
academic_level_Other          -0.278907
Name: Exam, dtype: float64
```

```
[29]: import statsmodels.api as sm
import pandas as pd
import numpy as np

# 1. Preparación de la base limpia
df_mco = pd.get_dummies(df_limpio, drop_first=True)

# 2. Definición de variables tras análisis de correlación (Evitando redundancia)
# Eliminamos sqrt_productivity por colinealidad extrema con productivity_score
X_vars_depuradas = [
    'productivity_score',      # Representa el motor de rendimiento
    'focus_index',           # Capacidad de concentración
    'burnout_level',          # Factor de riesgo negativo
    'online_classes_hours_index', # Impacto del entorno virtual
    'mental_health_score',     # Bienestar psicológico
    'study_hours'             # Esfuerzo directo
]

# Añadir dummies específicas que no estén altamente correlacionadas
cols_dummies = [c for c in df_mco.columns if 'gender_Male' in c or
    'academic_level_Undergraduate' in c]
X_final_list = X_vars_depuradas + cols_dummies

# 3. Construcción de Matrices
X = df_mco[X_final_list].astype(float)
X = sm.add_constant(X)
y = df_mco['Exam'] # Modelamos Asistencia

# 4. Ajuste del Modelo MCO Depurado
modelo_mco_sin_sesgo = sm.OLS(y, X).fit()

# 5. Resultados
print("--- MODELO MCO DEPURADO (SIN SESGO POR COLINEALIDAD) ---")
print(modelo_mco_sin_sesgo.summary())

# Verificación de Multicolinealidad (Opcional pero recomendado)
from statsmodels.stats.outliers_influence import variance_inflation_factor
vif_data = pd.DataFrame()
vif_data["Variable"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.
    shape[1])]
print("\nFactores de Inflación de la Varianza (VIF):")
print(vif_data)
```

```
--- MODELO MCO DEPURADO (SIN SESGO POR COLINEALIDAD) ---
```

OLS Regression Results

```

=====
Dep. Variable:          Exam    R-squared:                0.198
Model:                  OLS     Adj. R-squared:            0.197
Method:                 Least Squares    F-statistic:          159.9
Date:                  mié., 22 abr. 2026    Prob (F-statistic):    1.06e-241
Time:                  10:34:10    Log-Likelihood:        -1442.5
No. Observations:      5193    AIC:                   2903.
Df Residuals:          5184    BIC:                   2962.
Df Model:              8
Covariance Type:       nonrobust
=====

```

```

=====
                                coef    std err          t      P>|t|
-----
[0.025    0.975]
-----
const                0.3392     0.027    12.416     0.000
0.286    0.393
productivity_score    0.0052     0.001     6.895     0.000
0.004    0.007
focus_index          0.0046     0.001     6.356     0.000
0.003    0.006
burnout_level        -0.0024     0.000    -6.157     0.000
-0.003   -0.002
online_classes_hours_index  0.2648     0.017    15.774     0.000
0.232    0.298
mental_health_score   0.0015     0.003     0.498     0.619
-0.004    0.007
study_hours           0.0022     0.005     0.470     0.639
-0.007    0.011
gender_Male           0.0304     0.009     3.370     0.001
0.013    0.048
academic_level_Undergraduate  0.0535     0.010     5.572     0.000
0.035    0.072
=====

```

```

=====
Omnibus:              1158.368    Durbin-Watson:           2.022
Prob(Omnibus):        0.000    Jarque-Bera (JB):       2120.113
Skew:                 -1.422    Prob(JB):               0.00
Kurtosis:             4.308    Cond. No.               450.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Factores de Inflación de la Varianza (VIF):

Variable	VIF
----------	-----

```

0          const  37.905853
1  productivity_score  7.656257
2          focus_index  2.554928
3          burnout_level  1.469431
4  online_classes_hours_index  1.006351
5          mental_health_score  3.444127
6          study_hours  3.444226
7          gender_Male  1.004057
8  academic_level_Undergraduate  1.003882

```

```

[30]: import statsmodels.api as sm
import pandas as pd

# 1. Preparación de la base de datos
# Aseguramos que las dummies de 'Other' existan mediante get_dummies
df_mco = pd.get_dummies(df_limpio, columns=['part_time_job', 'gender'],
    drop_first=False)

# 2. Definición de las variables seleccionadas
# Usamos los nombres exactos que genera pandas para las categorías "Other"
X_vars = [
    'productivity_score',
    'part_time_job_Other',
    'gender_Other'
]

# 3. Construcción de matrices
# Verificamos que las columnas existan en df_mco antes de asignar
X = df_mco[X_vars].astype(float)
X = sm.add_constant(X) # Añadimos el intercepto (Beta 0)
y = df_mco['Exam']      # Variable dependiente: Asistencia

# 4. Ajuste del modelo MCO (OLS)
modelo_asistencia_reducido = sm.OLS(y, X).fit()

# 5. Resultados
print("--- MODELO PARSIMONIOSO: ASISTENCIA (3 VARIABLES) ---")
print(modelo_asistencia_reducido.summary())

```

--- MODELO PARSIMONIOSO: ASISTENCIA (3 VARIABLES) ---

OLS Regression Results

```

=====
Dep. Variable:          Exam    R-squared:                0.207
Model:                  OLS    Adj. R-squared:            0.207
Method:                 Least Squares    F-statistic:        452.3
Date:                   mié., 22 abr. 2026    Prob (F-statistic):    4.87e-261
Time:                   10:35:08    Log-Likelihood:        -1412.2
No. Observations:      5193    AIC:                    2832.

```

Df Residuals: 5189 BIC: 2859.
Df Model: 3
Covariance Type: nonrobust

	coef	std err	t	P> t	[0.025
0.975]					
const	0.5853	0.011	52.414	0.000	0.563
0.607					
productivity_score	0.0080	0.000	29.910	0.000	0.008
0.009					
part_time_job_Other	-0.3464	0.018	-19.084	0.000	-0.382
-0.311					
gender_Other	-0.0718	0.012	-5.949	0.000	-0.095
-0.048					
Omnibus:	1160.838	Durbin-Watson:	2.021		
Prob(Omnibus):	0.000	Jarque-Bera (JB):	2152.043		
Skew:	-1.407	Prob(JB):	0.00		
Kurtosis:	4.426	Cond. No.	170.		

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.